Минобрнауки России

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Волгоградский государственный технический университет»

Факультет электроники и вычислительной техники

Кафедра САПР и ПК

Согласовано

_____
(должность гл. специалиста предприятия)

_____ _____
(подпись) (инициалы, фамилия)

«_____» _____20___г

Утверждаю
Зав. кафедрой

_____ _____
(подпись) (инициалы, фамилия)

«_____» _____20___г

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к выпускной квалификационной работе бакалавра на тему
(наименование вида работы)
Разработка интеллектуального инструментария для
автоматизированного создания динамических игровых миров

Автор _____ Коваленко Кирилл Дмитриевич
(подпись и дата подписания) (фамилия, имя, отчество)

Обозначение _____
(код документа)

Группа ИВТ-465
(шифр группы)

Направление 09.03.01 Информатика и вычислительная техника
(код и наименование направления, наименование программы (профиля))

Профиль «Разработка игровых и XR приложений»

Руководитель работы _____ _____
(подпись и дата подписания) (инициалы и фамилия)

Консультанты по разделам:

_____ _____ _____
(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

_____ _____ _____
(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

_____ _____ _____
(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

Нормоконтролер _____ _____
(подпись, дата подписания) (инициалы и фамилия)

Волгоград 2027 г.

Минобрнауки России

Федеральное государственное бюджетное образовательное учреждение высшего образования
«Волгоградский государственный технический университет»

Кафедра САПР и ПК

Утверждаю
Зав. кафедрой

_____ _____
(подпись) (инициалы, фамилия)

«____»_____20____ г.

Задание на выпускную квалификационную работу бакалавра
(наименование вида работы)

Студент Коваленко Кирилл Дмитриевич
(фамилия, имя, отчество)

Код кафедры _____ Группа ИВТ-465

Тема Разработка интеллектуального инструментария для автоматизированного создания динамических игровых миров

Утверждена приказом по университету от «___» _____ 20___ г. № _____

Срок представления готовой работы _____
(дата, подпись студента)

Исходные данные для выполнения работы

Unity; локальные и бесплатные модельные API; авторский корпус знаний; процедурная спецификация игрового мира.

Содержание основной части пояснительной записки
1 Анализ процесса создания игрового мира
2 Проектирование инструментария
3 Реализация инструментария
4 Проверка и экспериментальная оценка

_____
_____
_____
_____
_____
_____
_____
_____
_____

_____

_____

_____

Перечень графического материала

1) Постановка задачи и требования

_____

2) Сопоставление подходов

_____

3) Архитектура инструментария

_____

4) Редакторский цикл и проверка

_____

5) Процедурный мир и навигация

_____

6) Знания и действия NPC

_____

7) Просмотр и компиляция C#

_____

8) Демонстрация продукта

_____

9) Методика испытаний

_____

10) Результаты и ограничения

_____

11) Заключение

_____

12) _____

_____

Руководитель работы _____ _____

(подпись и дата подписания) (инициалы и фамилия)

Консультанты по разделам:

_____ _____ _____

(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

_____ _____ _____

(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

_____ _____ _____

(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

_____ _____ _____

(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке WorldWeaver — инструментария Unity для создания динамических игровых миров по текстовому запросу. Обосновано разделение языкового планирования и проверяемых операций над сценой. Рассматриваются процедурный рельеф, индивидуальные знания NPC, игровые действия и просмотр программных изменений. Методика разграничивает программные инварианты, выполнение задания моделью и достоверность ответа. Сохранённые результаты относятся к явно указанным версиям продукта.

Ключевые слова: ИГРОВОЙ МИР, UNITY, ПРОЦЕДУРНАЯ ГЕНЕРАЦИЯ, ЯЗЫКОВАЯ МОДЕЛЬ, РЕДАКТОРСКИЙ АГЕНТ, НЕИГРОВОЙ ПЕРСОНАЖ, ИЗВЛЕЧЕНИЕ ЗНАНИЙ.

ABSTRACT

This bachelor thesis presents WorldWeaver, a Unity toolkit for creating dynamic game worlds from natural language requests. The architecture separates language model planning from verifiable scene operations. The thesis examines procedural terrain, individual NPC knowledge, game actions, and reviewed source code changes. The evaluation distinguishes software invariants, task completion, and answer accuracy. Previously collected results are attributed to their respective product versions.

Keywords: GAME WORLD, UNITY, PROCEDURAL GENERATION, LANGUAGE MODEL, EDITOR AGENT, NON PLAYER CHARACTER, KNOWLEDGE RETRIEVAL.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ..... 8
1 АНАЛИЗ ПРОЦЕССА СОЗДАНИЯ ИГРОВОГО МИРА.....11
1.1 Состав автоматизируемой деятельности.....11
1.2 Процедурная генерация и языковое планирование.....12
1.3 Знания и поведение персонажей.....14
1.4 Анализ существующих решений.....15
1.5 Постановка задачи и критерии результата.....18
2 ПРОЕКТИРОВАНИЕ ИНСТРУМЕНТАРИЯ.....20
2.1 Требования и границы автоматизации.....20
2.2 Разделение компонентов и выбор альтернатив.....23
2.3 Цикл выполнения задания и состояние работы.....25
2.4 Типизированный план и транзакция сцены.....29
2.5 Процедурная модель мира.....32
2.6 Знания и память NPC.....34
2.7 Игровые действия и программные изменения.....35
3 РЕАЛИЗАЦИЯ ИНСТРУМЕНТАРИЯ.....39
3.1 Структура пакета и среда.....39
3.2 Редакторский чат и управление заданием.....40
3.3 Исполнитель редакторских операций.....43
3.4 Генерация окружения и пространственные условия.....45
3.5 Провайдеры и обработка внешних отказов.....46
3.6 Диалоговый конвейер и игровые действия.....49
3.7 Исходный код и восстановление изменений.....52
4 ПРОВЕРКА И ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА.....54
4.1 Организация испытаний.....54
4.2 Программные и интеграционные проверки.....55
4.3 Влияние индивидуальных знаний.....56
4.4 Задержки и детерминированная генерация.....58
4.5 Модельные задания редакторского агента.....60

4.6 Проверка программного кода и восстановления.....62
4.7 Программные и предметные проверки редакции 1.2.....63
4.8 Приёмочные модельные прогоны редакции 1.2.....67
4.9 Ограничения вывода и дальнейшая проверка.....71
ЗАКЛЮЧЕНИЕ.....73
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....75
ПРИЛОЖЕНИЕ А Техническое задание.....81
А.1 Введение.....81
А.2 Основания для разработки.....81
А.3 Назначение разработки.....81
А.4 Требования к программе.....82
А.5 Требования к программной документации.....88
А.6 Технико-экономические показатели.....89
А.7 Стадии и этапы разработки.....89
А.8 Порядок контроля и приёмки.....89
А.9 Источники разработки.....92
ПРИЛОЖЕНИЕ Б Материалы повторения проверки.....94
Б.1 Установка и паспорт опыта.....94
Б.2 Программные и предметные проверки.....95
Б.3 Просмотр и компиляция исходника.....97
Б.4 Исторический корпус индивидуальных знаний.....98
Б.5 Ранние отрицательные наблюдения 1.2.....98
ПРИЛОЖЕНИЕ В Реестр доказательств.....100

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ

API — программный интерфейс.

LLM — большая языковая модель.

NPC — неигровой персонаж.

RAG — генерация ответа с извлечением внешних знаний.

UPM — менеджер пакетов Unity.

JSON — текстовый формат обмена структурированными данными.

UML — унифицированный язык моделирования.

NavMesh — навигационная сетка.

ВВЕДЕНИЕ

Создание игрового мира объединяет работу с рельефом, объектами, освещением, навигацией и поведением персонажей. В Unity эти элементы представлены разными компонентами и ассетами. Разработчик должен не только разместить объекты, но и согласовать их связи: персонаж должен знать допустимые цели движения, разговор должен учитывать доступные ему сведения, а передача предмета должна изменять действительные инвентари. Наличие отдельных генераторов и диалоговых систем не устраняет эту задачу интеграции.

Языковая модель позволяет задавать намерение обычным текстом. Например, запрос «создай лесную долину с деревней и торговцем» содержит как параметры окружения, так и требования к персонажу. Однако ответ модели не является готовым состоянием Unity. В нём могут отсутствовать обязательные поля, встречаться неизвестные объекты или действия, несовместимые с текущей сценой. Для практического применения требуется исполнитель, который проверяет предложенные операции, применяет допустимые изменения и сообщает наблюдаемый результат.

Актуальность работы определяется задачей соединения языкового интерфейса с управляемым игровым состоянием в повторно используемом инструменте. Современные решения уже предоставляют редакторских агентов, генераторы ассетов и интеллектуальных NPC. Поэтому разработка обосновывается конкретным процессом: подготовкой процедурного мира и персонажей в одном пакете, работой с локальными моделями и возможностью проверить изменения по данным сцены. Превосходство над всеми существующими средствами заранее не предполагается.

Объект исследования — автоматизация создания и изменения динамического игрового мира в среде Unity. Предмет исследования — способы объединения языкового планирования, процедурной генерации, индивидуальных знаний NPC и проверяемых операций над состоянием проекта.

Объект автоматизации — работа разработчика по подготовке окружения, персонажей и их взаимодействий.

Цель работы — разработать и оценить подключаемый инструментарий, преобразующий естественно-языковые задания в проверяемые операции создания игрового мира и контекстно-зависимые решения персонажей.

Для достижения цели необходимо решить следующие задачи:

— определить состав автоматизируемого процесса и сопоставить существующие подходы;

— сформулировать функциональные требования и ограничения изменения проекта;

— спроектировать взаимодействие чата, модели, типизированного плана и исполнителя;

— реализовать воспроизводимую генерацию рельефа и подготовку игровых объектов;

— связать индивидуальные знания NPC с диалогом, памятью и допустимыми действиями;

— обеспечить просмотр программных изменений, проверку компилятором и восстановление собственных файлов;

— провести программные и предметные испытания, раздельно оценив качество извлечения знаний и выполнение заданий агентом.

Исследовательская часть уточняется тремя вопросами. Первый: как индивидуальное извлечение знаний влияет на совпадение известного факта в ответе одной и той же локальной модели? Второй: какие задержки наблюдаются у выбранных локальных и бесплатной облачной конфигураций? Третий: какие инварианты сцены и игрового состояния удаётся обеспечить независимо от корректности вывода модели? Первые два вопроса исследуются на фиксированном корпусе вопросов, третий — с помощью программных и интеграционных испытаний. Полнота свободного редакторского задания оценивается отдельно по фактическому состоянию сцены.

Используются анализ первичных технических источников, объектное проектирование, процедурная генерация, векторный поиск, контрактная валидация и экспериментальное сравнение конфигураций. Работа имеет инженерный характер. Её вклад состоит в согласованной архитектуре и реализации процесса автоматизации, а не в предложении нового алгоритма языковой модели или нового метода генерации шума.

Практический результат предназначен для учебных и экспериментальных Unity-проектов. Код разделён на редакторскую и исполняемую части; модельный провайдер заменяется через общий интерфейс. Локальное выполнение уменьшает зависимость от платных запросов, но требует вычислительных ресурсов пользователя. Работа не доказывает сокращение трудозатрат начинающих разработчиков без отдельного пользовательского исследования и не приравнивает успешную компиляцию к безопасности произвольного кода.

Первый раздел содержит анализ предметной области, аналогов и постановку задачи. Во втором обосновываются требования, архитектура и границы доверия. Третий раскрывает реализацию инструментов и игровых подсистем. Четвёртый описывает методику, сохранённые экспериментальные результаты и ограничения их интерпретации. Полное техническое задание и материалы повторения проверки вынесены в приложения.

1 АНАЛИЗ ПРОЦЕССА СОЗДАНИЯ ИГРОВОГО МИРА

1.1 Состав автоматизируемой деятельности

Игровой мир представляет собой совокупность геометрии, объектов, правил и изменяющегося состояния. Процедурная генерация создаёт содержимое по формализованным параметрам и алгоритмам. В зависимости от задачи генерируются рельеф, уровни, расположение объектов, задания или отдельные элементы оформления [1, с. 1–2]. Для рассматриваемого инструментария принимается требование повторяемости: при одинаковых входных данных генератор должен создавать одинаковый результат. При этом большое количество объектов само по себе не обеспечивает связного игрового процесса.

В рассматриваемой задаче выделяются два этапа. На этапе подготовки проекта разработчик формирует окружение, назначает компоненты и задаёт персонажам знания и ограничения. Во время игры персонажи получают реплики и наблюдения, выбирают допустимые действия и меняют состояние мира. Для этих этапов нужны разные полномочия. Редакторский инструмент может создавать ассеты; runtime-персонажу достаточно ограниченного набора игровых действий. Предоставление обеим подсистемам неограниченного доступа к Unity API было бы необоснованным расширением области риска.

Обычный процесс создания небольшой сцены начинается с подготовки Terrain, материалов и освещения. Затем вручную размещаются здания и декор, задаются точки интереса, создаются NPC и их связи с объектами. После этого настраиваются диалог и навигация. Каждое изменение требует проверки: дерево может оказаться в воде, свет — под поверхностью, а цель патруля — за пределами доступного пути. Это объясняет, почему автоматизация должна проверять пространственные и предметные условия, а не ограничиваться сообщением об успешном вызове API.

Языковой интерфейс вводит дополнительный этап интерпретации. Пользователь формулирует задачу неполно: «сделай горы ниже» не содержит

ни идентификатора Terrain, ни нового значения высоты. Инструменту необходимо получить контекст текущего проекта, определить объект и предложить частичное изменение. Если вместо изменения создаётся второй мир, отдельная команда может быть технически корректна, но исходное задание останется невыполненным. Следовательно, единицей оценки должен быть запрос с его проверяемыми условиями, а не число исполненных команд.

Предметная модель включает разработчика, редакторский чат, активную сцену, ассеты, игровые сущности и внешнюю модель. Разработчик задаёт намерение и подтверждает программные изменения. Чат хранит ход работы. Исполнитель управляет явно разрешёнными операциями. Источником истины о результате являются компоненты, ассеты и игровые данные, прочитанные после исполнения. Текст ответа модели является предложением и пояснением, но не свидетельством существования объекта.

Для описания процесса «как есть» достаточно проследить один запрос на изменение. Разработчик определяет нужные объекты в Hierarchy, находит соответствующие параметры в Inspector, изменяет значения, запускает сцену и сопоставляет результат с намерением. Контекст удерживается самим разработчиком. При переходе к автоматизированному процессу действия не исчезают: часть из них выполняет инструмент, а контроль результата должен оставаться явным. Именно этот контроль связывает удобство естественного языка с требованиями к сохранности проекта.

1.2 Процедурная генерация и языковое планирование

Для рельефа пригодно представление полем высот. Оно совместимо со штатным Terrain Unity и позволяет независимо регулировать размер мира, перепад высот, масштаб шума и сглаживание. Метод ограничен поверхностью с одной высотой для каждой горизонтальной координаты: пещеры и нависающие скалы требуют иной геометрии. Для учебного открытого мира это приемлемый компромисс, поскольку поверхность можно использовать как источник высоты размещения и основу навигации [2] [1, с. 58–59].

Детерминированный генератор принимает точную спецификацию. Языковая модель может заполнить эту спецификацию по описанию, но не должна передавать массив высот или выполнять вычисления для каждой точки. Разделение сокращает размер модельного ответа и позволяет проверять параметры до создания ассетов. Оно также сохраняет возможность воспроизвести мир без повторного обращения к модели.

Размещение декора отличается от генерации поверхности. Для дерева проверяется высота над водой и допустимый уклон; для здания дополнительно важны размер площадки и свободное место. Псевдослучайное размещение с фиксированным seed позволяет повторять композицию. Однако детерминированность ещё не доказывает эстетическое качество. Оценка пригодности окружения требует самостоятельной рубрики: читаемости композиции, плотности объектов и доступности игровых маршрутов.

Прямая генерация C# по любому запросу является более универсальным путём, но переносит проверку на компилятор и человека. Типизированные инструменты покрывают меньшую область, зато позволяют точно определить предусловия и восстановление. Поэтому для повторяемых операций целесообразен встроенный исполнитель, а для нового поведения — отдельный процесс просмотра исходника. Эти способы дополняют друг друга; создание примитивов не равнозначно генерации нового художественного 3D-ассета.

Для многошагового выполнения важна обратная связь. Подход ReAct сочетает выбор действий с получением наблюдений от внешней среды [3]. В Unity наблюдением может быть перечень созданных объектов, фактические координаты или сообщение компилятора. Если следующий шаг опирается только на прежнее предположение модели, ошибка переносится дальше. Поэтому модель должна получать прочитанное состояние после исполнения и иметь ограниченную возможность исправить план.

Свободный текст и структурированные данные выполняют разные функции. Текст удобен для запроса и объяснения. Схема JSON задаёт допустимые поля, типы и диапазоны плана. Поддержка structured outputs

снижает число ответов неправильной формы, однако не заменяет предметную проверку: координата может иметь верный тип и всё же находиться под Terrain [4, 5]. Полнота исходного задания также не выводится из соответствия схеме.

1.3 Знания и поведение персонажей

Языковая модель не знает авторский лор нового игрового проекта. Передача всего текста мира в каждом запросе увеличивает контекст и смешивает знания персонажей. RAG сначала извлекает подходящие фрагменты из внешнего корпуса, а затем добавляет их к запросу модели. В исходной работе Lewis и соавторов retrieval соединяется с генерацией для задач, требующих внешних знаний [6]. В инструментальной системе этот принцип применим к именам, расписаниям, ценам и правилам вымышленного мира.

Для NPC существенна принадлежность знания. Страж может знать время открытия ворот, но не должен получать личную записку торговца только потому, что она близка вопросу по вектору. Следовательно, сначала проверяется область доступа, затем выполняется ранжирование. Это отличается от последующей просьбы к модели не разглашать уже полученный документ: ограничение доступа должно действовать до формирования контекста.

Постоянные знания, разговорная память и сведения о состоявшихся действиях имеют разное назначение. База знаний задаёт факты мира. Разговорная память помогает продолжить беседу. Эпизодическая запись сообщает, что действие уже произошло. Если прежняя просьба игрока попадёт в автономное планирование как новое поручение, персонаж может повторно отдать предмет. Поэтому источник записи и её статус должны сохраняться явно.

Работа Generative Agents показывает архитектурный интерес к памяти, извлечению и планированию поведения агентов [7]. Этот результат не означает, что достаточно добавить историю сообщений для получения убедительного игрового персонажа. В Unity дополнительно нужны права на действия, доступные цели и связь с действительным состоянием компонентов. Речь, предложение действия и результат исполнения необходимо различать.

Вызов модели каждый кадр непригоден для движения. Модель выбирает высокоуровневое намерение, например переместиться к дому, а NavMeshAgent выполняет поиск и прохождение пути. Кулдаун и запрет параллельного диалога ограничивают частоту запросов. Для простых повторяемых эффектов, таких как вращение, достаточно обычного runtime-компонента без LLM. Тем самым языковое планирование используется там, где требуется интерпретация контекста, а непрерывное управление остаётся детерминированным [8].

Особое место занимают последствия диалога. Фраза «возьми кристалл» должна быть сопоставлена с действием, количеством и получателем. Оба инвентаря изменяются только после проверки. Если модель обещала передачу, но операция отклонена, интерфейс обязан показать отказ. Иначе игрок получит противоречивые сведения, а разработчик не сможет отличить ошибку языка от ошибки состояния.

1.4 Анализ существующих решений

Сопоставление проводится по назначению компонентов и документированным возможностям. Оно не является очным испытанием производительности. Версии и условия внешних продуктов могут изменяться, поэтому результаты документального обзора относятся к 31 октября 2026 года. Подробные ссылки и границы проверенного материала сохранены в исследовательском каталоге проекта.

Unity Assistant предоставляет редакторский чат и операции над проектом. В документации версии 2.21 описаны режимы Ask, Plan и Agent, инструменты чтения и изменения сцены, работа со скриптами и визуальным контекстом. AI Gateway связывает редактор с внешними агентами [9, 10, 11]. Поэтому чат, просмотр плана или выбор уровня модели не являются самостоятельной новизной WorldWeaver. Бесплатные официальные MCP и CLI также не позволяют утверждать, что у экосистемы Unity отсутствует бесплатный путь автоматизации [12].

MCP for Unity предоставляет внешнему агенту инструменты взаимодействия с редактором [13]. Сильная сторона этого подхода —

использование уже существующей среды агента и широкого набора операций. Для собственной разработки выбран иной акцент: готовая связь между процедурным миром, индивидуальными знаниями NPC и игровыми действиями. Это специализация процесса, а не доказанное преимущество над универсальным агентом.

Convai документирует действия персонажей в Unity, включая настройку вызовов и поведения [14]. Прежний Inworld Character Studio закрыт; его возможности перенесены в Agent Runtime [15]. NVIDIA ACE объединяет технологии для цифровых персонажей [16]. Эти решения подтверждают, что разговорный NPC с внешними знаниями и действиями — существующий класс продуктов. Выбор собственного инструментария обосновывается контролем контрактов, локальным провайдером и прозрачностью конкретных проверок.

LLMUnity предоставляет интеграцию локальных языковых моделей в Unity [17]. Для WorldWeaver локальный inference вынесен в Ollama, чтобы редакторский код работал через HTTP-контракт и не зависел от конкретной встроенной библиотеки модели. Такой выбор упрощает замену модели, но добавляет внешний процесс и необходимость контролировать endpoint. LLMUnity остаётся содержательной альтернативой для проекта, которому важнее встроенное развёртывание.

Promethean AI и Procedural Worlds предлагают средства подготовки и размещения элементов окружения [18, 19]. Их наличие показывает, что создание геометрии и размещение объектов не следует оценивать только по возможностям языковой модели. В предлагаемом инструменте качество простого процедурного мира ограничено используемыми примитивами и правилами размещения. Генерация художественных ассетов, ригов и анимаций в область реализации не включается.

Сопоставление рассмотренных подходов приведено в таблице 1.

Таблица 1 — Сопоставление подходов к автоматизации Unity

Подход	**Основная область**	**Значение для принятого решения**
Unity Assistant и MCP	Универсальная автоматизация редактора	Чат и инструменты уже существуют; требуется обосновать специализацию
Convai, Inworld, ACE	Разговорные и интерактивные персонажи	Знания и действия необходимо сравнивать как отдельный процесс
LLMUnity и локальные серверы	Доступ к локальной модели	Выбирается заменяемый провайдер, учитывается внешний процесс
Процедурные генераторы	Создание окружения	Модель задаёт параметры, геометрию строит проверяемый алгоритм
WorldWeaver	Мир и NPC в одном переносимом пакете	Проверяются согласованность полного процесса и сохранность проекта

Из обзора не следует измеренное превосходство WorldWeaver по скорости, удобству или стоимости владения. Для такого вывода необходим одинаковый набор заданий и разрешений, повторные испытания аналогов и учёт ручной доводки. В данной работе сравнение документации используется для постановки требований и выбора архитектуры.

Университетские работы по процедурному окружению и Terrain подтверждают целесообразность анализа генератора как самостоятельного инструмента [20, 21, 22]. Исследование LLM для игровых NPC подчёркивает отдельную проблему оценки персонажей [23]. Эти работы используются как примеры постановки и методики, а не как источник чисел для собственного эксперимента. Наличие удачно защищённого аналога не подтверждает качество нового продукта без собственной проверки.

1.5 Постановка задачи и критерии результата

Требуется разработать UPM-пакет, который устанавливается в Unity-проект и предоставляет единый процесс создания мира и персонажей. Пользователь должен иметь возможность уточнить существующий объект, добавить знания NPC и проверить результат в Play Mode. Для часто используемых операций предусматриваются типизированные команды. Новый исходный код применяется отдельным процессом с просмотром и фактическим результатом компилятора.

Качество системы определяется несколькими независимыми свойствами. Корректность контракта означает, что план соответствует схеме. Корректность исполнения означает соблюдение предусловий и проверку результата по состоянию Unity. Полнота задания означает выполнение всех явно заданных условий. Содержательная достоверность речи требует оценки утверждений, включая дополнительные сведения, которых пользователь не запрашивал. Смешение этих уровней приводит к завышению результата: валидный JSON может описывать неверное действие, а успешный тест формата — пропустить подземный источник света.

В качестве исходных ограничений принимаются один активный Unity-проект, последовательное выполнение больших модельных запросов и конечный бюджет исправлений. Требование нулевых платежей за inference выполняется локальной конфигурацией и доступными бесплатными API без покупки токенов. Оно не равнозначно нулевой стоимости оборудования и не гарантирует будущую доступность бесплатного сервиса.

Для новой редакции предусматривается состояние задания, сохраняющее исходные требования и результаты этапов. Оно должно поддерживать продолжение после перезагрузки редактора, повторную проверку и отчёт о фактически выполненном. Долговременное хранение не должно превращать отвергнутый план в исполненный. Семантические места мира и маршруты NPC рассматриваются как явно именованные сущности, которые можно проверять независимо от формы геометрии.

Практический результат должен быть воспроизводимым: для каждой проверки фиксируются версия кода, модель, входное состояние и исход. Непройденные попытки сохраняются. Изменение критериев после обнаружения дефекта создаёт новый протокол, а не превращает прежний результат в успешный. Эта постановка определяет требования и архитектуру следующего раздела.

2 ПРОЕКТИРОВАНИЕ ИНСТРУМЕНТАРИЯ

2.1 Требования и границы автоматизации

В качестве формы поставки выбран UPM-пакет, устанавливаемый штатным Package Manager [24]. Инструментарий должен работать внутри существующего Unity-проекта и использовать активную сцену. Разработчик сохраняет обычный доступ к компонентам и ассетам; чат предоставляет дополнительный способ их подготовки. Для этого не требуется перенос игровых данных в отдельную закрытую среду или автоматическое удаление содержимого сцены.

Функциональные требования объединены в группы, соответствующие проверяемым результатам. Полные формулировки и условия приёмки приведены в техническом задании. Здесь рассматриваются решения, влияющие на архитектуру. Требования к новой редакции не считаются выполненными только из-за наличия записи в таблице: их подтверждение требует отдельного протокола.

Группы требований и наблюдаемые результаты приведены в таблице 2.

Таблица 2 — Группы требований и наблюдаемые результаты

Группа	**Требуемое поведение**	**Наблюдаемый результат**
Редакторский процесс	Запрос, план, исполнение, проверка и ограниченное исправление	Состояние задания и отчёт о применённых операциях
Создание мира	Рельеф, вода, окружение и именованные места	Компоненты сцены и сохраняемая спецификация

Группа	Требуемое поведение	Наблюдаемый результат
Персонажи	Личность, знания, память, допустимые действия и навигация	NPC с действительными ссылками и runtime-поведением
Изменение проекта	Явные цели, предконтроль и восстановление	Отсутствие непредусмотренных мутаций и доступная отмена
Программные изменения	Просмотр исходника и проверка компилятором	Различимые статусы записи, компиляции и подключения
Повторяемость	Фиксированные версии, входы и исходы	Машиночитаемые протоколы и повторяемые проверки

Основные нефункциональные требования связаны с сохранностью данных, ограничением ресурсов и наблюдаемостью. Большие модельные запросы выполняются последовательно. Число раундов и операций ограничивается; отмена не должна выдавать уже применённый результат за отсутствующий. Конфигурация модели хранится отдельно от логики генератора. Ключи внешних API не включаются в поставку и журналы.

Система не должна объявлять произвольный пользовательский проект полностью понятным по краткой выборке контекста. Отдельные инструменты чтения уточняют сведения перед изменением. Недостаток контекста должен приводить к уточнению или контролируемому отказу. Успешная компиляция подтверждает допустимость исходника для компилятора, но не отсутствие вредных операций и не правильность поведения. Эти ограничения определяют границы автоматизации.

Прецеденты разработчика и игрока показаны на рисунке 1. Разработчик управляет подготовкой проекта и просмотром исходников; игрок взаимодействует с уже созданным миром. Восстановление редакторской задачи и загрузка игрового сохранения относятся к разным участникам и механизмам.



Рисунок 1 — Прецеденты разработчика и игрока

2.2 Разделение компонентов и выбор альтернатив

Архитектура разделяет редакторский интерфейс, модельный провайдер, предметные сервисы и runtime. Сборка Editor зависит от UnityEditor и отвечает за создание ассетов, историю задания, Undo и работу со скриптами. Сборка Runtime содержит знания, инвентарь, действия, навигацию и взаимодействие NPC. Runtime не должен ссылаться на UnityEditor, чтобы редакторский интерфейс не становился обязательной частью игры.

Рассмотрены три способа выполнения пользовательского запроса. Первый — возвращать только текстовую инструкцию. Он минимально затрагивает проект, но оставляет всю работу пользователю. Второй — генерировать C# для любой операции. Он расширяет область задач, однако затрудняет предварительную проверку и восстановление. Третий — выбирать типизированные инструменты, а новый код вынести в отдельный просматриваемый процесс. Для основной реализации выбран третий способ: частые операции имеют явные предусловия, а новая программа не импортируется как побочный эффект обычного плана.

Для хранения знаний можно использовать внешний векторный сервис либо индекс в памяти. Внешняя база удобна при большом корпусе и нескольких клиентах, но добавляет развёртывание и сетевую зависимость. Для ограниченного учебного мира выбран локальный индекс с заменяемым интерфейсом. При росте корпуса допустима другая реализация без изменения диалогового конвейера. Библиотеки LangChain и LlamaIndex демонстрируют распространённость разделения индекса, retrieval и генерации, однако их полный стек для пакета Unity не требуется [25, 26].

Для локального inference выбран отдельный Ollama-процесс. Альтернатива — встроить модель через LLMUnity или llama.cpp [17, 27]. HTTP-граница упрощает смену модели и позволяет использовать одну серверную установку для редактора и NPC. Её цена — запуск дополнительного процесса, ожидание загрузки модели и обработка сетевых ошибок даже на loopback. Эта цена учитывается в методике измерения полного времени.

Компоненты соединяются через небольшие контракты: CompletionRequest и CompletionResult для генерации, embedding-интерфейс для векторов, EditorPlan для редакторских операций, NpcDecision для игровых действий. Предметный код не должен знать особенности полей конкретного облачного API. Это позволяет испытывать ошибочные ответы через управляемый провайдер и проверять реальную модель отдельно.

Распределение обязанностей между компонентами показано на рисунке 2. На схеме сервисы объединены по роли: контекст проекта включает индекс, выбранные навыки и заметки, а проверка и продолжение объединяют независимый контроль условий и координатор возобновления. История задачи хранится внутри истории чата, отдельная база заданий не вводится.

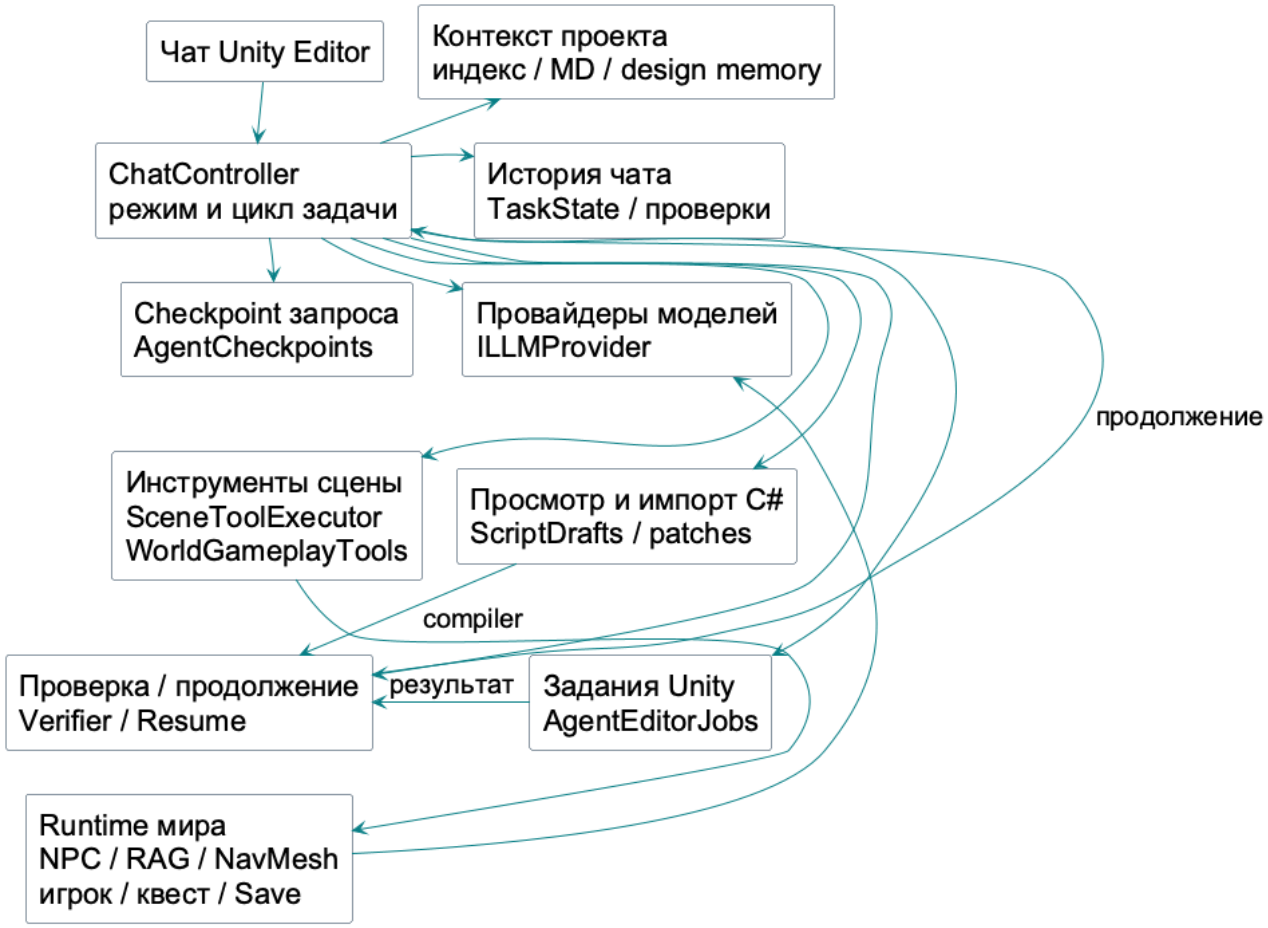


Рисунок 2 — Компоненты редакторского агента и игрового мира

Процессы и хранилища показаны на рисунке 3. Модель обслуживается отдельным локальным процессом либо необязательным облачным API. История редактора и журналы импорта находятся в UserSettings, создаваемые

ассеты — в проекте, игровое сохранение — в отдельном каталоге persistentDataPath. Эти границы позволяют различать восстановление проекта, повторную загрузку модели и загрузку состояния игры.

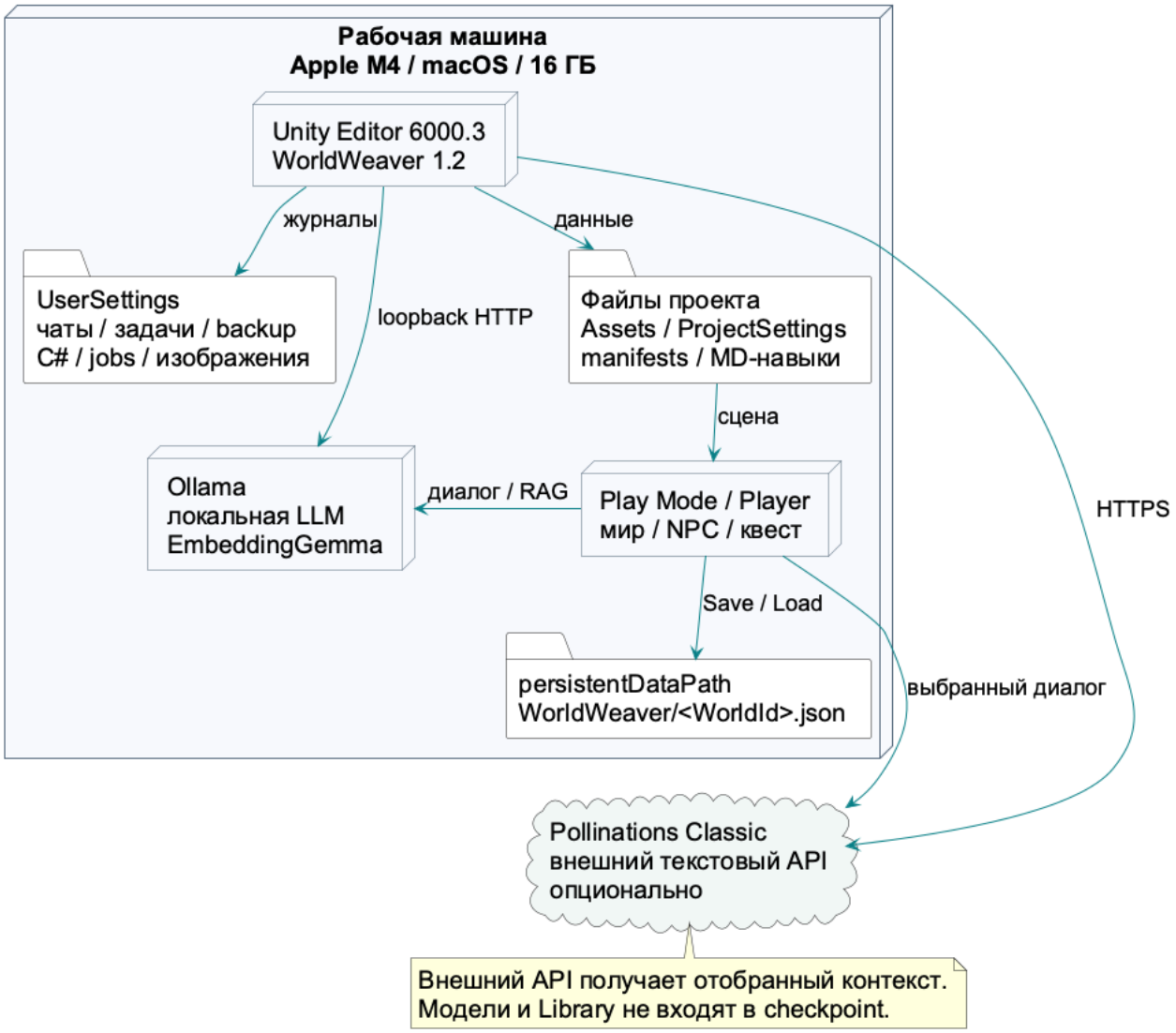


Рисунок 3 — Процессы и хранилища инструментария

2.3 Цикл выполнения задания и состояние работы

Процесс «как будет» начинается с сохранения исходного запроса. Затем собирается ограниченный контекст проекта, выбираются необходимые инструкции и вызывается модель. Полученный план проверяется целиком. После применения операций исполнитель читает фактическое состояние; модель получает результаты и сопоставляет их с первоначальной целью. Если остаются нарушения и бюджет, допускается исправление. Завершение фиксируется отдельным отчётом, а не одним полем complete в первом ответе.

В редакции 1.2 используется AgentTaskState — сохраняемое состояние задания. В нём должны различаться требования пользователя, предложенные шаги, применённые изменения, результаты проверки и причины остановки. Запись «план предложен» не означает «шаг исполнен». После перезагрузки домена состояние должно позволять понять, какой импорт уже начался, получен ли результат компилятора и какой следующий шаг допустим. Это особенно важно для C#, поскольку Unity может прервать обычный ход редакторского кода перекомпиляцией.

Режимы Ask, Plan и Execute разделяют полномочия. Ask допускает чтение и объяснение. Plan формирует предложение без изменения сцены. Execute допускает встроенные операции в пределах разрешений и бюджета. Запрос пользователя только объяснить состояние должен ограничивать последующие действия независимо от предположения модели. Программные изменения дополнительно требуют просмотра конкретного исходника; общий режим Execute не заменяет эту процедуру.

Checkpoint обращения разделяет охват резервного копирования и охват восстановления. Перед изменением копируются файлы Assets, ProjectSettings, manifest и lock-файл пакетов, а также рабочие состояния открытых сцен. Это широкий, но ограниченный снимок: не более 20000 файлов, 300 МиБ с учётом сцен и от одной до восьми загруженных сцен. Library и внешнее хранилище весов модели в него не входят. Ограничение исключает применение данного механизма к произвольному проекту любого размера.

После выполнения фиксируется WriteSet — пути, действительно изменённые обращением. Файловое восстановление затрагивает только это множество; перед ним проверяются сохранённые версии и конфликты последующих правок. Рабочие состояния сцен проверяются отдельно. Undo пакета команд и журнал версий исходников дополняют этот механизм, но не заменяют дисковый checkpoint. Сведения о незавершённой операции сохраняются, поскольку обычная память C# недоступна после перезагрузки домена.

Каталог узких инструкций проектируется как дополнительный источник правил выполнения. Вместо передачи всех документов в каждый запрос выбирается небольшое число подходящих навыков. Для внешнего материала фиксируются лицензия, исходный URL и commit. Такая инструкция может рекомендовать API, но не получает прав на исполнение и не заменяет валидатор. Её применимость проверяется по установленной версии Unity; содержимое прикреплённых документов рассматривается как данные, если пользователь не дал соответствующего поручения.

Ограниченный цикл исправления нужен по двум причинам. Во-первых, ошибочный результат может требовать нового чтения сцены. Во-вторых, бесконечное повторение повышает стоимость и вероятность лишних мутаций. Поэтому после исчерпания бюджета возвращается частичный результат с незавершёнными условиями. Если последний раунд применил операцию, но отдельная модельная проверка уже невозможна, интерфейс должен сообщить именно о выполненном шаге, а не о доказанном завершении всего задания.

Основные контракты сохраняемого задания представлены на рисунке 4. Диаграмма выбирает классы, определяющие состояние обращения и связь плана с исполнением; она не является полным перечнем классов пакета. Состояние ожидания компиляции сохраняется отдельно от факта успешного выполнения.

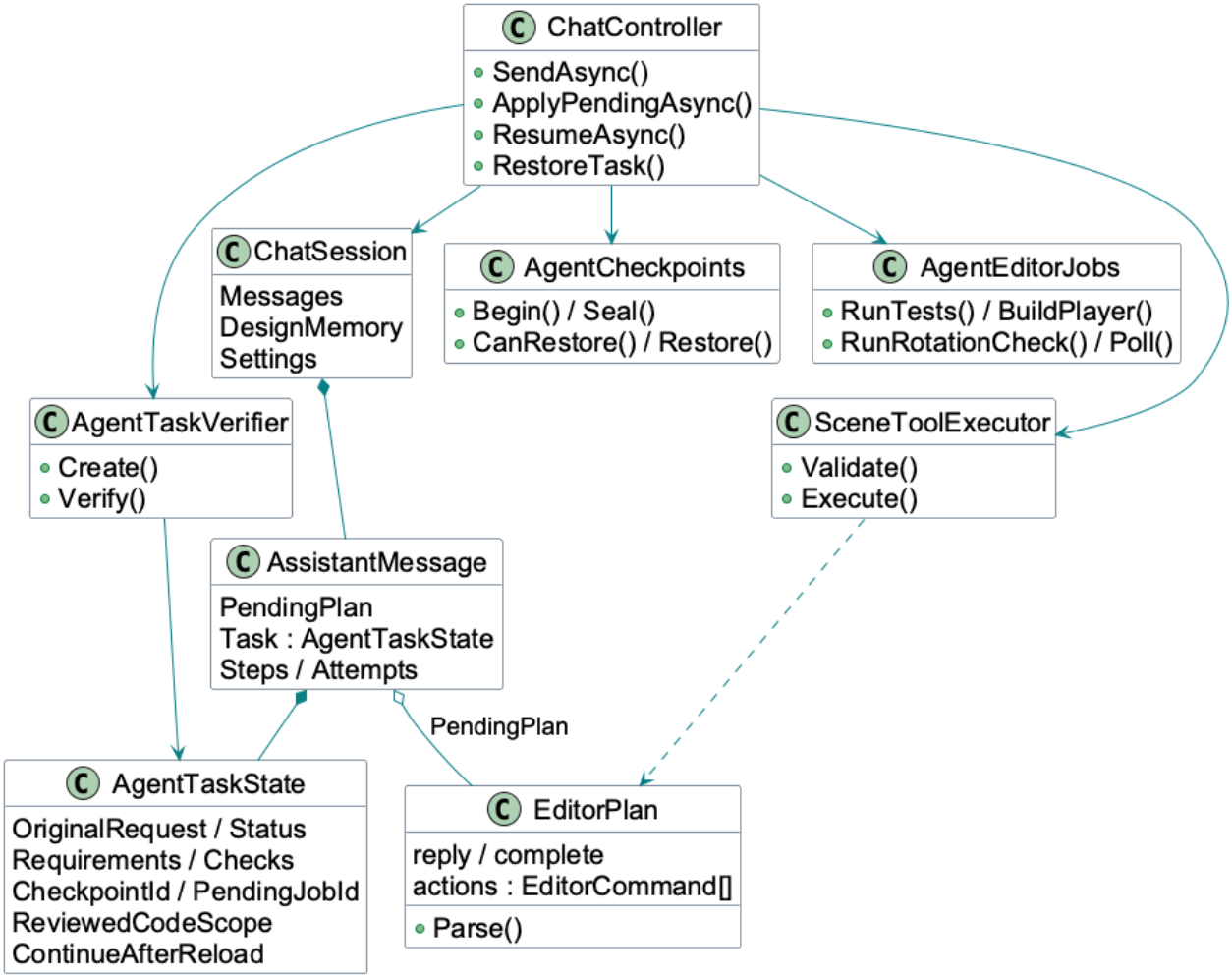


Рисунок 4 — Основные контракты задачи и редакторского плана

Полный цикл показан на рисунке 5. Новые предложения проверяются по разрешениям. Программное изменение вне ранее просмотренной области снова требует просмотра; узкий ремонт внутри согласованной области выделяется в отдельный ограниченный случай. Конечная проверка читает ограниченный набор явно распознаваемых условий: она не доказывает полноту произвольного описания пользователя. Ожидание завершения Unity и повторное обращение к модели учитываются как этапы одного задания.

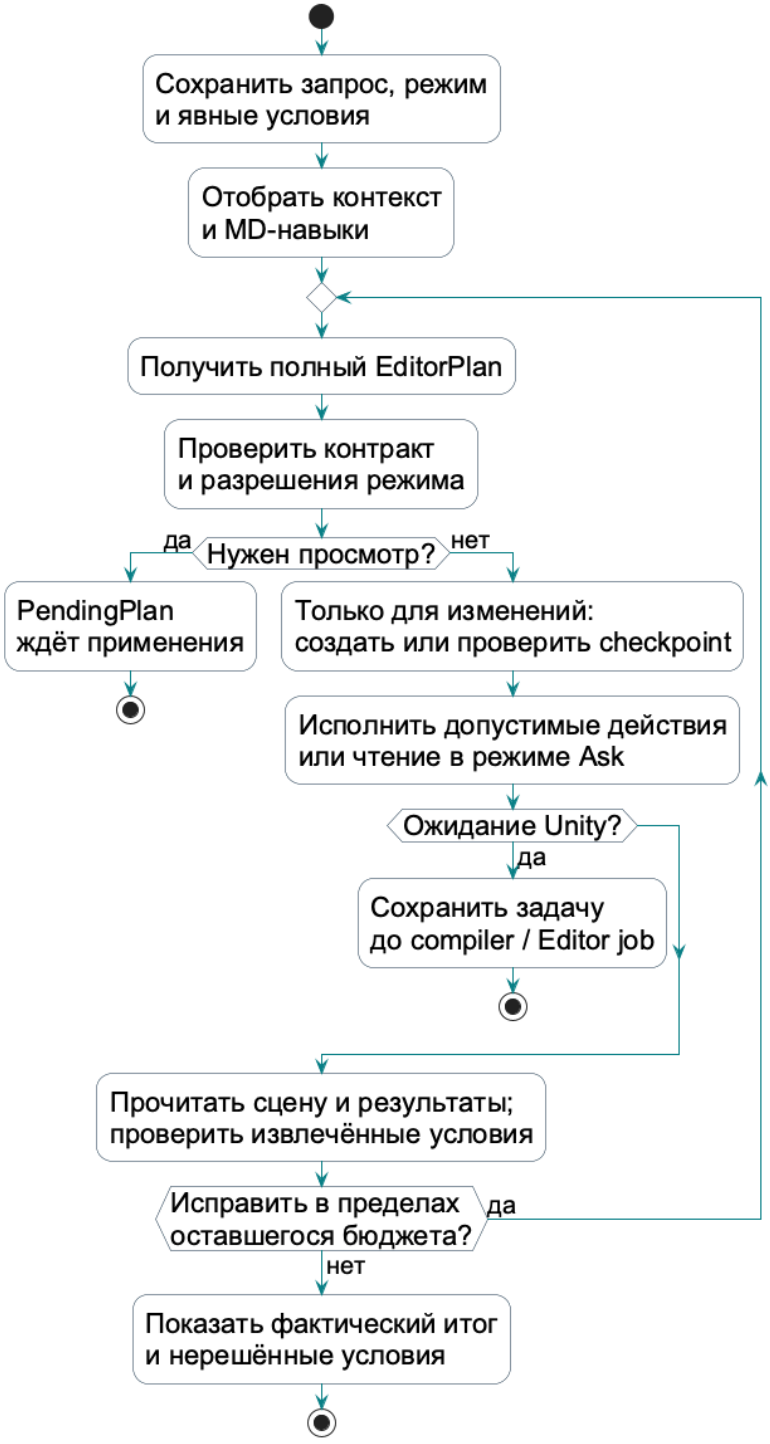


Рисунок 5 — Цикл выполнения и проверки редакторской задачи

2.4 Типизированный план и транзакция сцены

EditorPlan содержит текст ответа, признак завершения и список EditorCommand. Команда определяет инструмент, имя, цель и объект аргументов. Для каждого инструмента задаются допустимые ключи и типы. Неизвестное поле отклоняется: неявное игнорирование опечатки вроде heigth создало бы результат, отличающийся от намерения пользователя. Числа

проверяются на конечность и диапазон, цвета — на поддерживаемый формат, пути — на принадлежность разрешённому каталогу.

Прикладная схема и схема для декодера имеют разные назначения. Полная схема описывает требования инструмента. Конкретный сервер может поддерживать только часть JSON Schema. Для него формируется совместимое представление, но полученный экземпляр всё равно проходит полный прикладной контроль. Ограничения преобразования схем в грамматики документированы в llama.cpp; успешное constrained decoding не является доказательством семантической корректности плана [28].

Для явно распознаваемых условий схема уточняется под конкретное обращение. Например, при создании мира 160 × 160 м с ровно 12 деревьями, 3 домами и 4 камнями эти значения становятся обязательными допустимыми значениями аргументов. Отдельный хост-контроль проверяет то же ограничение и повторное размещение декораций, поэтому облачный сервер без полной поддержки схемы не получает исключения. Привязка высоты требует подходящего предмета: рост NPC не должен превращаться в высоту Terrain. Неоднозначные числа не связываются с первым попавшимся полем. Это конечный набор распознаваемых условий, а не универсальное понимание любого русского предложения.

Уточнение запроса на новый мир использует общий конечный предикат выбора инструментов и схемы. Слово «долина» как место размещения NPC или света само по себе не означает создания нового Terrain. Такое разделение сохраняет обычную ёмкость пакета при редактировании существующей сцены; иначе ограничение первого макродействия необоснованно запрещало бы три источника света. Нераспознанная формулировка следует обычному выбору инструментов и не считается автоматическим отказом.

Аналогично, распознанное поручение изменить только цель NPC ограничивает update_npc полем goals и не означает настройки движения персонажа. Составной запрос сохраняет отдельно запрошенные инструменты. Эти правила применяются и схемой, и хост-контролем. Числовые границы всё

равно проверяются исполнителем: поддержка JSON Schema сервером не считается гарантией отклонения каждого недопустимого значения при генерации.

Перед мутацией проверяется весь пакет. Если поздняя команда ссылается на отсутствующий объект, ранние команды не должны частично изменить сцену. При этом допускается ссылка на объект, создаваемый предыдущей командой того же плана. Для этого валидатор строит проектируемое множество целей и проверяет зависимости в порядке команд. Такая предварительная модель не подменяет фактического read-back после исполнения.

В сцене используются явные идентификаторы целей. Имена удобны человеку, но могут повторяться. Instance ID пригоден в пределах текущего редакторского сеанса; после reload требуется устойчивый идентификатор проекта. Смена способа адресации не должна незаметно выбирать первый одноимённый объект. Неоднозначность должна обнаруживаться до изменения.

Созданные инструментом объекты имеют маркер происхождения и собственную группу. Удаление ограничивается собственными объектами. Явно выбранный пользовательский объект допускается изменять только разрешённым инструментом с Undo; принадлежность группе не даёт права изменять произвольные соседние объекты. Ограничения количества элементов действуют как на отдельный пакет, так и на накопленную сцену.

Успешный пакет объединяется в одну группу Undo [29]. При исключении сцена откатывается, а созданные в этом пакете ассеты удаляются по точному журналу путей. Чужие файлы не удаляются по одному лишь совпадению имени. Чтение объектов и поиск проекта не должны создавать фиктивную группу отмены. Сохранение сцены является отдельным явным инструментом; оно не выполняется после каждого модельного ответа.

Read-back содержит только подтверждённые сведения: целевой объект, применённые параметры и созданные пути. Для источника света читаются тип, цвет, интенсивность и положение относительно поверхности. Для NPC —

компоненты, роль, знания и ограничения. Это позволяет проверять условия следующего шага, не полагаясь на свободную строку Summary.

Потоки данных редакторского агента показаны на рисунке 6. Предложение модели и результат проверки представлены разными потоками. Журнал Before/After и множество записанных файлов связывают восстановление с конкретной операцией; модель не получает права произвольно перезаписывать остальные данные проекта. Показанный поток проверки не означает, что любой текстовый запрос уже имеет исчерпывающий автоматический критерий.

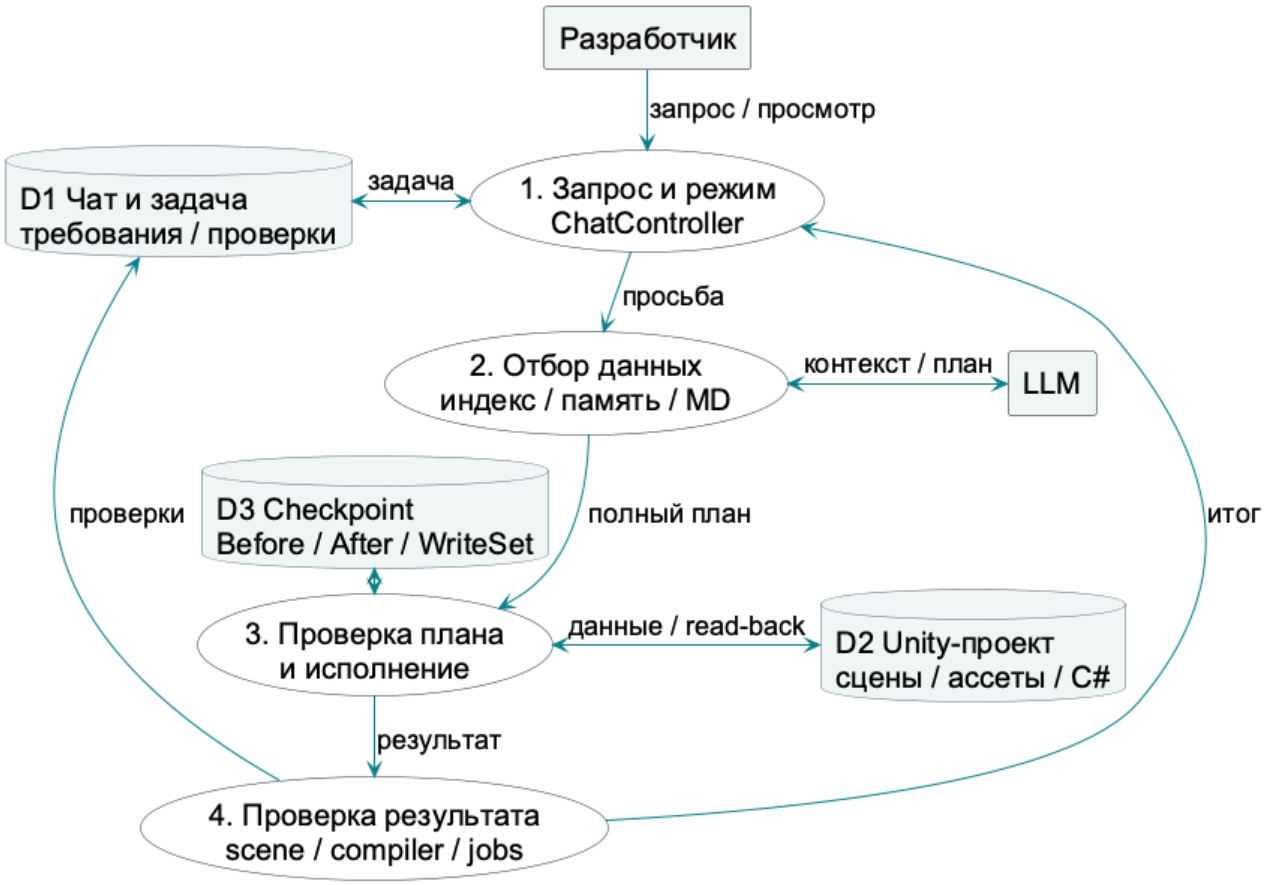


Рисунок 6 — Потоки данных редакторского агента

2.5 Процедурная модель мира

TerrainSpec хранит размер поверхности, разрешение heightmap, максимальную высоту, seed и параметры шума. Размеры задаются в единицах Unity, рассматриваемых в демонстрации как метры. Разрешение выбирается из допустимых значений Terrain. Модель заполняет параметры, генератор строит нормированное поле и передаёт его в TerrainData.SetHeights [2].

Для точки поверхности используется нормированная сумма шумовых октав. Каждая октава имеет амплитуду a и частоту f, a seed определяет смещение. Нормирование удерживает результат в одном диапазоне при изменении числа октав. Формула генератора имеет вид:

$$F(u,v) = \frac{\sum_{o=0}^{O-1} a_o N(f_o u, f_o v, s_o)}{\sum_{o=0}^{O-1} a_o}$$

Здесь N — используемая шумовая функция, O — число октав, u и v — нормированные горизонтальные координаты. Эта формула является выбранным способом реализации известного процедурного приёма, а не новым алгоритмом. Дополнительная маска позволяет менять форму края мира. Сглаживание применяется ограниченным числом проходов и также входит в спецификацию.

Частичное изменение объединяется с сохранённым TerrainSpec. Если запрос меняет только высоту гор, остальные параметры остаются прежними. После объединения снова проверяется вся спецификация. Материалы поверхности должны пересчитываться вместе с высотами, иначе визуальный слой будет соответствовать прежней форме. Изменение рельефа также может сделать ранее корректное размещение объектов неверным; это отдельное условие проверки, а не автоматически решённое свойство update_world.

Обычный create_npc при включённом on_terrain вычисляет начальную высоту по Terrain и ограниченно ищет сухое место; явный отказ сохраняет заданные координаты. Эта операция сама по себе не создаёт постоянной привязки персонажа. Иной механизм используется для отмеченных объектов семантической деревни и новых Point/Spot, размещённых относительно Terrain: WorldSurfaceAnchor сохраняет ссылку на конкретную поверхность и смещение. Для источника учитываются земля, вода и зазор не менее 3 м. При update_world отмеченные объекты повторно выравниваются по своему Terrain; произвольные неотмеченные объекты такой гарантии не получают.

Деревья, камни, дома и NPC собираются из примитивов с материалами. У дерева выделяются ствол и крона, у дома — стены, крыша и проём. Это обеспечивает различимость элементов без внешних платных ассетов, но ограничивает художественное качество. Для одного render pipeline нельзя выбирать материал другого только по наличию shader в проекте: выбор должен соответствовать активному pipeline.

В редакции 1.2 предусматривается семантическое представление мест, дорог, ворот и входов. Геометрическое имя объекта недостаточно для задания «патрулируй между воротами и площадью»: необходимо хранить связь имени с положением и доступностью места. NavMesh строится по допустимой геометрии, а именованный маршрут связывается с проверенными точками. Доступность пути подтверждается построением и движением, а не наличием массива координат в компоненте.

2.6 Знания и память NPC

KnowledgeDocument содержит идентификатор, заголовок, текст и область доступа. Текст делится на перекрывающиеся фрагменты, чтобы извлечение не теряло факт на границе окна. Вектор строится выбранной embedding-моделью. Для исходного эксперимента использована EmbeddingGemma [30, 31]. При изменении модели или размерности индекс необходимо перестроить; векторы разных пространств не сопоставляются.

Для ранжирования применяется косинусное сходство:

$$c(q,d) = \frac{\sum_{i=1}^{D} q_i d_i}{\sqrt{\sum_{i=1}^{D} q_i^2} \cdot \sqrt{\sum_{i=1}^{D} d_i^2}}$$

Здесь q — вектор вопроса, d — вектор фрагмента, D — размерность. После фильтрации доступа выбираются первые k фрагментов. Для небольшого корпуса полный перебор имеет линейную сложность по числу векторов и остаётся простым для проверки. Переход к приближённому индексу оправдан при росте корпуса, но требует отдельной оценки полноты поиска.

Формирование контекста начинается с правил роли и ограничений. Затем добавляются доступные знания, фактическое состояние и подходящие записи памяти. Документ может содержать инструкции, противоречащие системным правилам. Поэтому извлечённый текст не получает полномочий сам по себе. Даже если модель последует такой инструкции, исполнитель всё равно проверяет действие по текущему состоянию.

Память ограничивается по объёму. Временная разговорная запись и подтверждённый эпизод различаются типом. Персистентное хранение в новой редакции должно связывать запись с персонажем и миром, чтобы загрузка другого сохранения не приносила чужие эпизоды. Успешный Save/Load проверяется по дисковому состоянию и восстановленным данным, а не по тексту уведомления.

Ошибку обновления памяти следует отделять от уже состоявшегося игрового действия. Если предмет передан, а индексация эпизода отменена, нельзя возвращать общий статус, создающий впечатление отсутствия передачи. Результат изменения инвентаря сохраняется, а ошибка памяти указывается отдельно. Это правило определяет границу фиксации операции.

2.7 Игровые действия и программные изменения

NpcDecision содержит речь и список игровых действий. Реестр IGameAction разрешает только зарегистрированные типы. Для передачи проверяются item, target, положительное количество, остаток, вместимость и дальность. При нескольких передачах строится проектируемый остаток, чтобы пакет не мог потратить один и тот же предмет дважды. Отказ должен происходить до первой передачи.

Навигационные действия проверяют состояние NavMeshAgent и полноту пути. Модель выбирает цель, а исполнитель вычисляет конкретное перемещение. Наличие MoveTo в JSON не считается движением. Для проверки используются положение, состояние агента и достижение целевой области. Аналогично настройка patrol не доказывает прохождение маршрута: требуется наблюдение во времени.

Создание нового C# отделяется от обычных действий сцены. Сначала пользователь просматривает точный исходник и целевой путь. Затем файл записывается и Unity запускает настоящий импорт. Событие компилятора определяет результат. Подключение MonoBehaviour разрешается после подтверждения соответствия импортированного типа исходнику. Иначе устаревший тип из прежней сборки может быть ошибочно принят за результат нового файла.

Для восстановления ведётся отдельный журнал каждого импорта: состояние до записи, состояние после записи, метаданные и идентификатор сообщения. Второй импорт не перезаписывает резервную копию первого. Восстановление допускается только при совпадении текущего файла с ожидаемой версией после импорта. Это защищает ручные правки и позволяет последовательно вернуть B, затем A при перекрывающихся изменениях. Проверка компилятора после восстановления обязательна.

Продолжение после reload ограничено тремя попытками. Автоматический ремонт допускается только в первых двух продолжениях, в ранее просмотренном файле, при совпадении его текущего hash с записанным результатом и наличии действительной последней ошибки компилятора. Поддерживаются два узких случая: точка с запятой CS1002 в указанной позиции либо замена неизвестного имени CS0103 единственным явно заданным числовым литералом исходного запроса. Иные изменения требуют нового просмотра. Это не универсальное автоматическое исправление C#.

Просмотр и hash не создают sandbox. Код MonoBehaviour исполняется с полномочиями среды Unity и может иметь произвольные эффекты. Поэтому импорт нового модельного исходника требует проверки человеком; отдельные стандартные действия предпочтительны для уже поддерживаемых задач. В отчёте должны различаться отказ просмотра, ошибка записи, ошибка компиляции и ошибка runtime.

Последовательность просмотра исходника, импорта, компиляции и продолжения показана на рисунке 7. После reload координатор читает

сохранённый результат. Новый исходник и изменение за пределами согласованного ремонта вновь требуют предусмотренного просмотра. Сохранение состояния ожидания не гарантирует автоматический перезапуск задания, прерванного закрытием редактора.

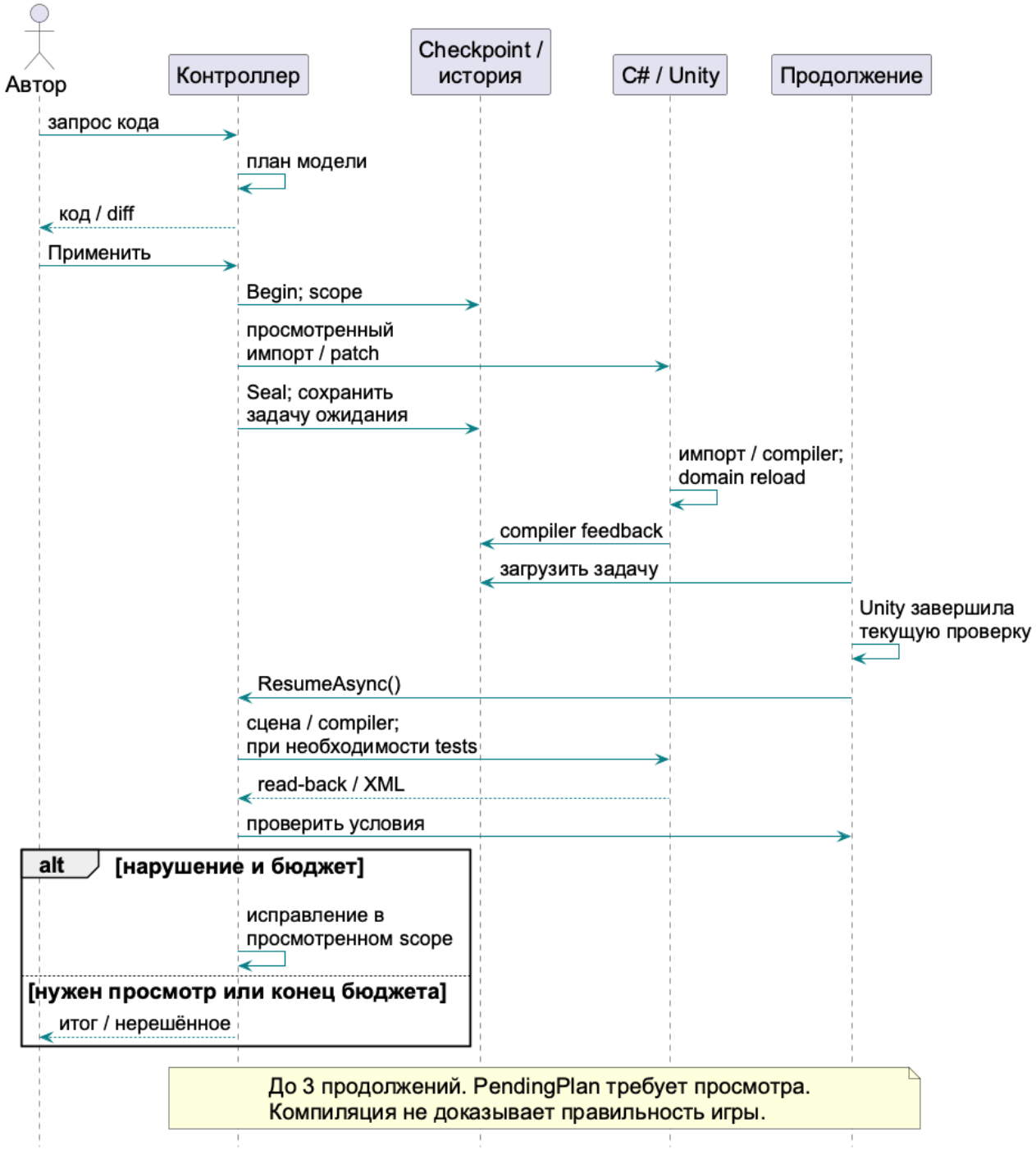


Рисунок 7 — Просмотр C#, компиляция и продолжение задачи

Таким образом, выбранная архитектура сохраняет модель как источник плана, а проверяемые сервисы — как исполнителей. Предлагаемые изменения новой редакции расширяют длительность и связность задания через

сохраняемое состояние, семантический мир и продолжение после reload. Подтверждение этих свойств относится к испытаниям соответствующей версии.

3 РЕАЛИЗАЦИЯ ИНСТРУМЕНТАРИЯ

3.1 Структура пакета и среда

WorldWeaver реализован как UPM-пакет с отдельными сборками Runtime, Editor и Tests. Runtime содержит провайдеры, генератор, знания, инвентарь и обработчики действий. Editor добавляет чат, создание ассетов, просмотр исходников и средства исследования. Разделение зафиксировано asmdef-файлами. Редакторская сборка не включается в исполняемую игру, а runtime-компоненты доступны без открытого окна инструментария.

Основная среда разработки и сохранённых испытаний редакторского агента — Unity 6000.3.27f1 [32]. Испытательная машина: Apple M4 с 16 ГиБ объединённой памяти, macOS 15.7.4. Базовое ядро ранее проверялось в Unity 2022.3. Этот результат относится к исходной версии и не подтверждает совместимость всех последующих редакторских функций с Unity 2022. Для любого нового протокола версия пакета и commit фиксируются отдельно.

Подключение из Git или tarball использует штатный Package Manager [24]. Сценовые компоненты и данные не зашиваются в тестовый проект: пакет может создать необходимые собственные ассеты в проекте-потребителе. Чистая установка должна подтверждаться новой Library и реальными ссылками, иначе успешная работа может зависеть от оставшихся локальных файлов разработчика.

Профиль OrchestratorProfile задаёт маршруты Terrain, Dialogue и Planning и embedding-провайдер. Отдельная настройка редакторского чата определяет его модель и бюджет. Это позволяет использовать небольшую модель для одного типа задачи и другую для свободного планирования, не меняя обработчики инвентаря или генератор. Маршрутизация не выполняет скрытую подмену ответа при недоступности сервера: ошибка отражается в trace.

3.2 Редакторский чат и управление заданием

Основной интерфейс — чат на UI Toolkit. В нём задаются запрос, модель и режим работы; рядом выводятся этап и время ожидания. Меню добавления контекста связывает задание с выбранными объектами и материалами проекта. История показывает модельные попытки, вызванные инструменты и результаты. При уменьшении ширины заголовок переносится, карточки размещаются в один столбец, а история открывается отдельной панелью. Область приветствия прокручивается независимо от поля ввода. Пять специализированных разделов прежнего окна доступны как Advanced Inspector и используются для точной настройки Terrain, знаний, маршрутов и диагностики.

Основное окно показано на рисунке 8. Снимок иллюстрирует расположение органов управления; наличие карточки задания не доказывает успешность его модельного исполнения.

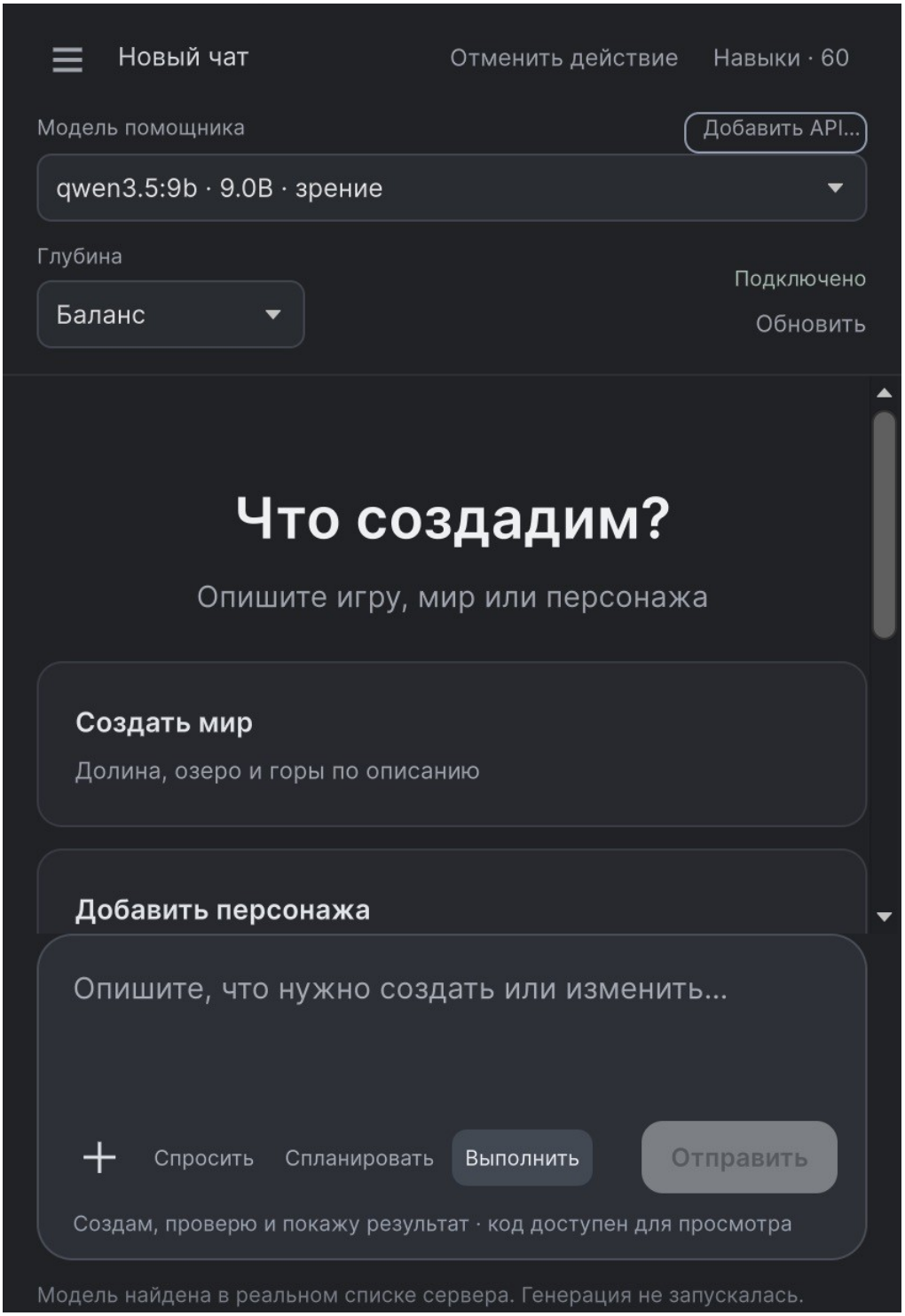


Рисунок 8 — Фрагмент редакторского чата WorldWeaver 1.2 в узком окне

ChatController получает исходную цель, собирает свежий контекст и вызывает ILLMProvider. Ответ разбирается как EditorPlan. Неизвестные ключи, повторённые свойства, неверные типы и превышение размера отклоняются. После прикладной проверки исполнитель возвращает EditorBatchResult. Для следующего раунда передаются фактические результаты, а не строка модели о том, что действие якобы завершено.

Поле complete не прекращает процесс сразу после мутации, если бюджет позволяет проверку. Модель получает read-back и должна сопоставить его с исходным заданием. Пустой список действий с завершением может служить итоговым ответом. Если последний разрешённый раунд только применил шаг, статус ограничивается подтверждённой операцией. В управляемом тесте такая схема позволяет сначала создать объект, затем обнаружить неверное положение и исправить его следующим инструментом.

Режим «Спросить» (Ask) запрещает мутации и новый код. «Спланировать» (Plan) сохраняет предложение для просмотра. «Выполнить» (Execute) допускает поддерживаемые инструменты в пределах разрешений. Независимо от выбранного режима явная просьба только объяснить состояние ограничивает операции чтением. Валидация права на изменение выполняется до применения; нельзя полагаться только на соответствующий текст системного prompt.

Настройки мощности ограничивают число раундов. Они представляют бюджет обработки, а не измеренный рейтинг качества модели. Более длинный цикл может исправить ошибку, но увеличивает время и число возможностей незапрошенного изменения. Контроллер ограничивает общее число вызовов и размер возвращаемого контекста. Одновременный запуск нескольких больших моделей на машине с 16 ГиБ памяти не используется как способ ускорить одно задание.

Состояние Unity, полные исходные требования и результаты инструментов остаются на стороне хоста. Для модели строится ограниченная проекция: типовые деревья, камни и дома агрегируются, но выбранные, явно названные и входящие в область задания цели сохраняют идентичность. Бюджет сцены составляет 3000, 4000 или 6500 символов в зависимости от режима мощности. Если обязательные идентичности не помещаются, переполнение обозначается явно. Сокращение не даёт права считать пропущенный объект отсутствующим: find_scene_objects и inspect_object

уточняют данные. Критерии выполнения и восстановление используют полные внутренние данные, а не сокращённый текст модели.

История сохраняется в UserSettings отдельно от пакета. Записи включают исходный ответ и, при применении ограниченного адаптера, нормализованную форму с указанием преобразования. Повреждённая история не должна молча заменяться пустой. Известные шаблоны credentials маскируются перед отображением и записью; это ограниченный фильтр, поэтому прикрепляемый материал требует осмысленного выбора.

В редакции 1.2 управление длительным заданием использует сохраняемое состояние требований, этапов и проверок. AgentTasks и AgentCheckpoints разделяют текущее обращение, применённые шаги и продолжение после reload. Каталог содержит 60 узких Markdown-навыков. Материал адаптирован из двух MIT-проектов: Unity Claude Skills и Unity-Skills; версии upstream закреплены commit, а исходные hash, авторство и тексты лицензий сохранены в пакете [33, 34]. В модельный запрос автоматически выбирается до трёх подходящих рабочих инструкций, а не вся библиотека; дополнительные материалы читаются отдельным инструментом. Навыки не выполняются как скрипты и не меняют разрешения пользователя. Программные проверки этих механизмов и отдельный опыт реального reload приведены в разделе 4.7; модельная успешность оценивается самостоятельно.

3.3 Исполнитель редакторских операций

SceneToolExecutor предоставляет контекст, инструкции инструментов, предконтроль плана и пакетное исполнение. В базовом редакторском наборе поддерживаются создание и изменение мира, NPC и объектов, размещение множества элементов, Transform, материалы, освещение, камера, встроенное поведение, сохранение сцены и чтение проекта. Новый код обрабатывается отдельным сервисом ScriptDrafts. Контекст ограничен по числу и объёму объектов, поэтому инструменты inspect_object, search_project и read_script уточняют сведения по запросу.

Валидатор проверяет все команды до первой мутации. Для create_world допустимы частичные параметры с определёнными defaults; неизвестные аргументы запрещены. Для set_transform проверяются конечные значения координат и масштаба. Для чтения скрипта проверяются каталог Assets, расширение .cs и предел размера. Произвольная загрузка типа через reflection не используется как универсальный инструмент.

Объекты получают GeneratedSceneObject с семантическим видом tree, house, rock или иным поддерживаемым значением. Маркер помогает отличить логический объект от его составных частей и ограничивает удаление собственными объектами. Обычные пользовательские объекты изменяются только по явной цели. Если имена неоднозначны, исполнитель не должен угадывать нужный экземпляр.

При успешном пакете используется одна группа Undo. При ошибке Undo возвращает состояние сцены, а точный список созданных ассетов служит основанием для очистки. Read-only пакет не создаёт группу отмены. Сохранение выполняется только по инструменту save_scene; для новой сцены используется допустимый собственный путь. Автоматическое сохранение всей пользовательской работы после каждого ответа исключено.

Создание NPC добавляет IntelligentNPC, Inventory и WorldEntity, настраивает KnowledgeBase и профиль маршрутов. Изменение цели существующего NPC не должно заменять его KnownEntities или заново инициализировать чужие связи. Созданные и обновлённые персонажи обрабатываются отдельно: первоначальное окружение назначается новым, а сохранение существующих ссылок проверяется регрессией.

Результат инструмента читается из Unity. Для NPC это назначенные данные и компоненты, для Terrain — сохранённая спецификация, для света — его тип и параметры. Понятная короткая строка интерфейса сопровождается структурированной диагностикой. Таким образом, надпись «создано» может быть сопоставлена с действительным объектом и путём ассета.

3.4 Генерация окружения и пространственные условия

Генератор вычисляет heightmap в обычном C# по TerrainSpec и фиксированному seed. Нормированные значения передаются в TerrainData. Пресеты задают исходную комбинацию параметров, а пользовательские значения проходят тот же контроль. Смена seed изменяет поверхность, повтор с прежними параметрами воспроизводит массив в одной проверенной среде.

WorldComposer создаёт Terrain, воду, материалы, освещение и камеру, затем размещает деревья, камни и здания. Высота берётся из поверхности. Поиск места ограничен числом попыток, чтобы недоступный участок не создавал бесконечного цикла. Количество создаваемых объектов учитывает составные части: одно дерево включает несколько примитивов, а дом — стены, крышу и детали. Ограничение только числа логических деревьев не ограничило бы действительный размер Hierarchy.

Материал выбирается с учётом активного render pipeline. Наличие URP shader в установленных пакетах не означает использование URP текущим проектом. Для Built-in применяется совместимый материал; Terrain также получает подходящий shader. После изменения формы поверхности пересчитываются слои, чтобы визуальные зоны не оставались от предыдущего рельефа.

Для первоначального размещения обычным create_npc по умолчанию ищется сухая точка рядом с запрошенным положением. Явный режим без привязки к Terrain сохраняет заданные xyz и пригоден для специальных сцен. Этот способ сам по себе не добавляет постоянный anchor. Персонажи, создаваемые как часть семантической деревни, получают WorldSurfaceAnchor и рассматриваются отдельно при изменении рельефа.

Новые Point и Spot при размещении относительно Terrain получают зазор над землёй и водой и сохраняемый WorldSurfaceAnchor. Привязка указывает конкретный Terrain, в том числе при нескольких мирах, и участвует в повторном выравнивании. Для распознаваемого запроса о новых Point в существующем мире независимый критерий дополнительно проверяет

количество, границы поверхности, высоту и достаточный range. Его конечная грамматика не охватывает любой свободный запрос: явные xyz, подземное размещение и другие специальные указания сохраняют обычный контракт. Общий set_lighting проверяет типы и диапазоны параметров, но сам по себе не доказывает освещение земли. Само наличие трёх Light поэтому недостаточно.

Сохраняемая привязка имеет явный срок действия. Отказ on_terrain:false, преобразование в Directional, новая позиция существующего света без выравнивания либо отсутствие покрывающего Terrain удаляют прежний anchor. Изменение только цвета или интенсивности сохраняет его. Это предотвращает последующее неожиданное перемещение источника при update_world; переходы проверяются также через Undo и сохранение сцены.

Геометрия собирается из штатных примитивов. В поставке нет скрытой покупки ассетов или подмены сгенерированного результата готовой коммерческой сценой. Этот выбор делает происхождение содержимого проверяемым, но требует сдержанной оценки визуального результата. Для публикации изображения сцены сохраняются как реальные скриншоты Editor; схемы интерфейса не выдаются за работающий продукт.

Редакция 1.2 связывает геометрию с именованными местами и навигацией. WorldNavigation хранит связь мест, входов и маршрутов; построенный NavMesh используется для проверки полного пути. Закрытые ворота влияют на доступность прохода. Дорога, вход или ворота поэтому рассматриваются как условия перемещения, а не только названия GameObject. Проверка одного лишь поля PatrolPoints недостаточна: в Play Mode отдельно измеряется фактическое движение.

3.5 Провайдеры и обработка внешних отказов

ILLMProvider отделяет формирование запроса от сетевого API. Локальный Ollama поддерживает структурированные ответы [4]. OpenAI-compatible адаптер использует Chat Completions и поддерживаемую сервером форму response_format [35]. Совместимость протокола не гарантирует поддержки всех полей и возможностей конкретной модели.

Кнопка «Добавить API…» открывает форму с типом протокола, понятным названием, URL сервера и точным ID модели. Для OpenAI-compatible соединения выбирается JsonSchema, JsonObject или Text. В последнем режиме поле response_format не отправляется, но ответ всё равно проходит прикладной парсер и проверку плана. Родные Anthropic Messages и Gemini этой формой напрямую не поддерживаются. Пользовательское OpenAI-compatible подключение принимает только текст; локальный Ollama допускает изображения после подтверждения возможностей через /api/show.

Несекретные сведения сохраняются в UserSettings/WorldWeaverModels/connections.json. Сам ключ, введённый в форму, помещается в окружение процесса Unity: он сохраняется при domain reload и исчезает при закрытии процесса. Другой вариант — заранее настроенная пользователем переменная окружения, имя которой записывается в конфигурацию. Удаление подключения очищает созданный пакетом сеансовый ключ, но не внешнюю пользовательскую переменную. Удалённый endpoint должен использовать HTTPS; незашифрованный HTTP разрешён только для loopback.

«Проверить соединение» читает список моделей; для Ollama дополнительно уточняются возможности модели. Генерация при этом не запускается. Только отдельная команда «Проверить ответ» отправляет запрос с пределом 64 выходных токена и может расходовать квоту внешнего API. Пример подключения локального Ollama через совместимый путь /v1 показан на рисунке 9. Положительный статус на снимке подтверждает чтение списка моделей, а не выполнение игрового задания.

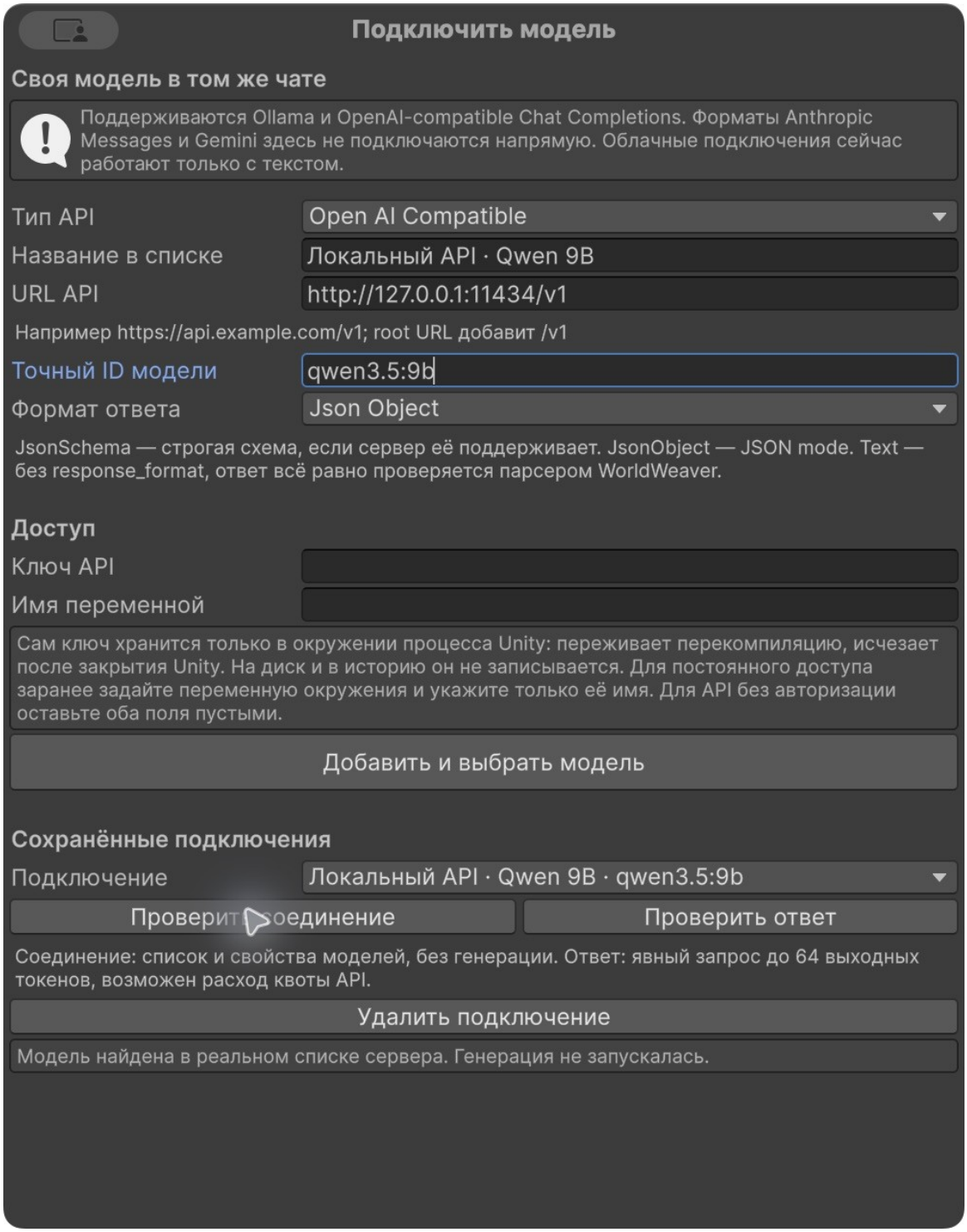


Рисунок 9 — Форма подключения API и результат проверки списка моделей

Локальная конфигурация редакторского агента использует qwen3.5:9b как стартовую модель. Это настройка выбранной машины, а не универсальная рекомендация для любого компьютера. Исходный QA-эксперимент использовал Qwen3.5 2B [36], а отдельные Planning-запросы — Qwen2.5-Coder. Изменение default не меняет модель уже сохранённого эксперимента.

Бесплатный Pollinations Classic рассматривается как внешний ограниченный сервис. В проверенном pinned исходнике сервера предусмотрены 45 секунд между бесплатными запросами одного IP и предел max_tokens 1500; клиент использует запас до 46 секунд [37]. Общая APIDOCS содержит другое историческое значение, поэтому в паспорте опыта фиксируются действительная конфигурация и версия адаптера [38]. Старые измерения версии 1.0 с интервалом 31 секунда сохраняются как исторические.

Повторы допускаются только для распознанных временных отказов. Classic FREE_LIMIT может требовать ожидания даже без Retry-After; для него используется ограниченный известный интервал. Обычный неверный запрос не повторяется бесконечно. Timeout и cancellation возвращают контролируемый исход. Провайдерская ошибка не превращается в скриптовый запасной ответ NPC.

Для Classic предусмотрено узкое исправление формы ответа: исходный текст всегда сохраняется отдельно, а нормализация явно отмечается. Такая мера не должна исправлять внутреннюю ошибку произвольного JSON или ослаблять запрет инструмента. Ollama не использует этот адаптер. Проверки сырого ответа, преобразованного экземпляра и разрешений остаются раздельными.

Потоковый вывод и визуальные вложения новой редакции требуют отдельного учёта. Появление текста в интерфейсе не означает завершения структурированного плана, а модель без поддержки vision не должна получать изображение как будто оно обработано. В итоговом отчёте необходимо фиксировать возможности выбранного провайдера и фактически переданный контекст.

3.6 Диалоговый конвейер и игровые действия

Диалог начинается с роли NPC и текущего состояния. Retrieval отбирает доступные фрагменты KnowledgeBase, затем добавляются подходящие записи памяти. Общий бюджет ограничивает объём контекста. При несовместимой размерности векторов требуется переиндексация; отмена перестроения не должна уничтожать уже пригодный индекс.

NpcDecision.Parse выполняет строгий разбор речи и списка действий. Повторённые поля, неверный тип количества и неизвестные имена отклоняются. Действия поступают в ActionDispatcher. Только после принятия и исполнения интерфейс показывает подтверждённый результат. Поле speech не используется для изменения инвентаря.

GiveItem проверяет разрешение, цель, item, расстояние, положительное количество, остаток и вместимость. При пакете действий учитываются все запрошенные изменения. TakeItem дополнительно требует права на изъятие. Наличие обработчика в реестре само по себе не предоставляет персонажу это право.

Для исходной демонстрационной площади DemoNavigation строит навигационные данные через NavMeshBuilder.BuildNavMeshData [39]. MoveTo, Follow и Flee используют NavMeshAgent. В Follow направление обновляется обычным runtime-кодом без нового вызова LLM на каждом кадре. Planning запускается по cooldown и не параллельно активному разговору. Его разрешения являются подмножеством общих действий NPC, чтобы автономный цикл не получил право передачи предметов из прежней реплики игрока.

Укрупнённая модель состояний NPC показана на рисунке 10. Для читаемости Idle и Waiting объединены в один узел, как и Moving и Following; значения перечисления остаются различными. Переход по достижении цели относится к Moving. Схема описывает управление состоянием, а не частоту успешных решений языковой модели.

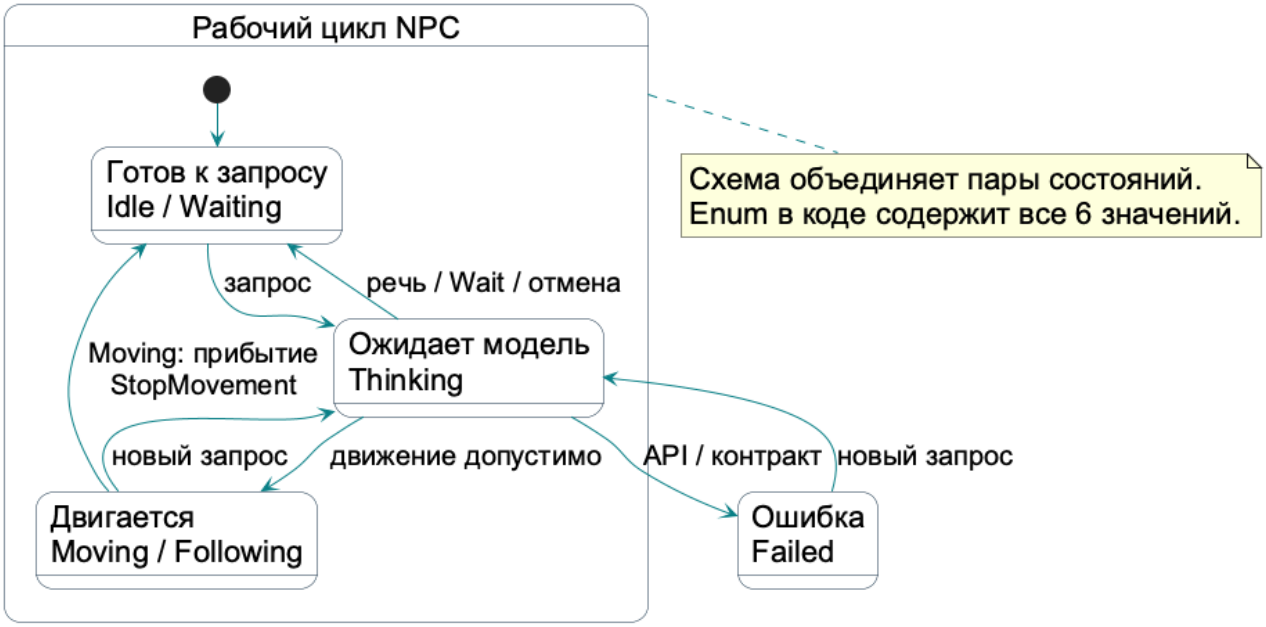


Рисунок 10 — Укрупнённые состояния интеллектуального персонажа

SceneBehavior реализует простые rotate, bob и другие заранее определённые эффекты. Эти компоненты следует отличать от интеллектуального планирования. В исходной редакции патруль по Transform-точкам не являлся автоматическим обходом препятствий по NavMesh. Именованный патруль новой редакции использует NavMesh; обход препятствия и отказ при недостижимом пути проверены самостоятельными PlayMode-сценариями.

WorldPlayground позволяет проверить созданного NPC в Play Mode без ручной привязки к старому демонстрационному интерфейсу. При отсутствии Player создаётся минимальный игровой участник, а персонажи находятся по фактическим компонентам. Показанная реплика проходит реальный TalkAsync. Удачное единичное наблюдение через GUI остаётся наблюдением и не заменяет выборку модельных запросов.

Игровой слой 1.2 включает управляемого игрока, взаимодействие, доставку предмета и однократную награду. WorldClock задаёт время, а распорядок NPC выбирает именованные места с учётом интервалов и доступного пути. Дисковое состояние объединяет инвентарь, этап доставки и эпизоды нужного мира. Повторное завершение после Save/Load не должно

удваивать награду. Эти инварианты проверяются по данным компонентов; фактические PlayMode-результаты отделены от ответов модели.

3.7 Исходный код и восстановление изменений

ScriptDrafts отделяет подготовку исходника от импорта. Сообщение хранит точный текст, путь и привязку к цели. Кнопка применения действует только на актуальный ожидающий план; исторический снимок после импорта или возврата доступен для чтения и не создаёт новый импорт. Такое разделение предотвращает случайное повторное применение старого кода.

Импорт может вызвать reload. Поэтому сервис сохраняет ожидание компиляции и проверяет настоящее событие Unity. Запись файла, успешная компиляция и подключение компонента имеют разные статусы. Перед attach проверяются класс MonoBehaviour и соответствие импортированного исходника. Устаревший GetClass до завершения нового импорта не считается успехом.

Каждый импорт имеет собственный журнал. Проверка текущего hash перед восстановлением защищает последующие ручные изменения. Для перекрывающихся импортов A и B допускается порядок возврата B, затем A. Попытка вернуть A поверх ещё действующего B отклоняется. После удаления нового исходника или восстановления прежнего .meta сохраняется нужная идентичность ассета, а результат повторной компиляции фиксируется.

Редакция 1.2 поддерживает не только новый файл, но и просмотренные изменения существующего исходника. Требование к ним строже: до записи нужен точный снимок исходного файла и проверка, что пользователь не успел изменить его после предложения. Автоматическое разрешение конфликтов с непроверенным содержимым не допускается. Путь, diff и результат компиляции должны оставаться связанными с конкретным обращением.

Встроенный исполнитель не является средой изоляции произвольного C#. Проверенный hash доказывает, какой текст был согласован, но не доказывает его безопасность. Изолированная статическая компиляция полезна

как ранний контроль API, однако не заменяет импорт Unity, подключение и runtime-проверку. Эти этапы в эксперименте оцениваются раздельно.

Таким образом, реализация соединяет языковой интерфейс с конкретными объектами и файлами проекта. Механизмы 1.0 и 1.1 образуют основу, а изменения 1.2 имеют отдельные паспорта и протоколы. В следующем разделе программные результаты новой редакции отделяются от исторических модельных опытов и текущего нового inference.

4 ПРОВЕРКА И ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА

4.1 Организация испытаний

Проверка разделена на три уровня: программные инварианты, выполнение конкретного модельного задания и содержательная оценка результата. Управляемый провайдер воспроизводит ошибочный ответ и позволяет проверить отсутствие побочного эффекта. Реальная модель подтверждает фактическую интеграцию, но единичный успех не определяет вероятность успеха на произвольной задаче. Визуальный просмотр дополняет чтение компонентов и не подменяет его.

Для каждого опыта фиксируются версия кода, Unity, модель, исходное состояние, параметры, критерий и полный исход. Ранние неудачные попытки не удаляются. Если после независимого анализа меняется критерий, новый запуск получает собственное имя. В частности, проверка наличия трёх источников света и проверка трёх источников над поверхностью являются разными критериями.

Сохранённые данные относятся к нескольким этапам. Базовый QA-эксперимент 1.0 изучает индивидуальные знания NPC. Протоколы 1.1 проверяют редакторский цикл, выполнение семи последовательных заданий и импорт исходника. Для редакции 1.2 отдельно сохранены испытания состояния задания, восстановления, семантических маршрутов, игрового цикла и поставки; модельные разговоры рассматриваются независимо от программных проверок. Результаты прежних версий не переатрибутируются новой реализации.

Программные тесты выполнены средствами Unity Test Framework [40]. XML показывает количество и исходы проверок, но не качество свободного текста. JSON-протокол сценария показывает действительное состояние сцены, но его автоматический критерий также может быть неполным. Поэтому существенные результаты сопоставляются с независимой проверкой исходных сообщений, компонентов и файлов.

4.2 Программные и интеграционные проверки

Исходный EditMode-набор ядра в Unity 6 включает 56 проверок. Для Terrain проверяются диапазоны, обязательные поля и повторяемость массива. Для инвентаря — обе стороны передачи, количество, вместимость, расстояние и суммарный бюджет. Для NpcDecision проверяются отсутствующие, лишние и повторённые поля. Retrieval испытывается на ранжировании, области доступа и несовместимой размерности векторов.

Сетевые проверки используют локальный TCP-сервер с управляемыми ответами. Он возвращает временный отказ, клиентскую ошибку, неверный JSON или задерживает ответ. Такой способ позволяет проверять retry, timeout и cancellation без намеренного расходования внешней квоты. Внешний inference испытывается отдельно и не выводится из положительного mock-теста.

Для редакторского агента набор расширен. В зафиксированном checkpoint 60192a9 от 31.10.2026 EditMode завершился с результатом 108/108: 51 проверка Core, 8 Transport, 27 SceneTool и 22 Controller; ошибок и пропусков нет, длительность 8,953 с. Этот набор подтверждает перечисленные программные свойства редакции 1.1. Более ранний PlayMode-протокол checkpoint 0aa9bdf содержит 3/3 успешных проверки за 1,057 с. Числа не складываются в метрику модельной успешности.

Среди значимых регрессий — отклонение последней неверной команды до мутации всего плана, одна группа Undo, очистка новых ассетов при ошибке, сохранение KnownEntities существующего NPC, запрет изменения в read-only запросе и ограничение нормализации Classic. Контроллер проверен на многошаговом исправлении по реальному read-back: прежнее complete не отменяет следующую сверку цели. При остановке после фиксации действия результат не маскируется общей ошибкой отмены.

Исходный сквозной runtime-сценарий использует реальный диалог и последующее Planning. Начальный инвентарь торговца содержит десять Crystal, игрока — ноль. После принятого GiveItem ожидаются девять и один. Затем измеряется фактическое перемещение NavMeshAgent к House. Протокол

ограничен тремя заранее заданными переговорными репликами; запасного скриптового GiveItem при отказе модели нет.

Состояние инвентарей и измеренное перемещение приведены в таблице 3.

Таблица 3 — Результаты исходного сквозного runtime-сценария

Маршрут исходного сценария	**Ходов до передачи**	**Crystal NPC / Player**	**Перемещение**
Локальный	2	9 / 1	11,6803 м
Облачный Dialogue и локальный Planning	1	9 / 1	11,6437 м

Финальный интеграционный XML содержит два успешных теста за 85,16 с. Для каждого маршрута это один ограниченный прогон. Дополнительный локальный GUI-повтор после передачи вернул допустимые пустые actions и не выполнил перемещение. Следовательно, два успешных теста не доказывают гарантированного завершения при повторении. Также обнаружено несогласование речи с GiveItem: осторожная реплика сохранилась, хотя передача уже произошла. Состояние поэтому читается из Inventory.

4.3 Влияние индивидуальных знаний

QA-эксперимент содержит 30 вопросов к авторскому лору с известными ответами. Проверены три конфигурации: Qwen3.5 2B без RAG, та же модель с RAG и GPT-OSS 20B с тем же retrieval. Всего сохранено 90 запросов: 89 ответов модели и одна ошибка HTTP 400. Ошибка остаётся в знаменателе

качества. Накопление памяти между вопросами отключено, чтобы сравнение характеризовало статическую базу знаний.

Метрика факта определяется наличием допустимого нормализованного варианта ответа в speech. Регистр и ё приводятся к общей форме, для чисел учитываются границы последовательности. Это proxy совпадения известного факта, а не экспертная проверка всех утверждений. Правильное имя может соседствовать с неверным пояснением. Поэтому дополнение доли до единицы нельзя называть частотой галлюцинаций.

Отдельно оценивается факт в принятой реплике, действительно доступной игроку после валидации. Если модель назвала правильный факт, но предложила недопустимое действие, исходный и пользовательский результаты различаются. Для retrieval Hit@3 означает присутствие размеченного документа среди первых трёх статических фрагментов; MRR@3 учитывает позицию первого правильного источника.

Результаты фиксированного корпуса приведены в таблице 4.

Таблица 4 — Совпадение факта, проверка действий и извлечение знаний

Показатель	**Qwen3.5 2B без RAG**	**Qwen3.5 2B с RAG**	**GPT-OSS 20B с RAG**
Факт в исходной речи	2/30	20/30	29/30
Факт в принятой реплике	1/30	16/30	16/30
JSON прошёл парсер	30/30	30/30	29/30
Пакет действий принят	27/30	22/30	16/30
Hit@3	0/30	30/30	30/30
MRR@3	0,000	0,983	0,983
Ошибка API	0	0	1

Вариант без RAG не выполняет извлечение. Нули Hit@3 и MRR@3 в этой колонке обозначают отключённый поиск, а не измеренную неспособность поискового алгоритма найти документ.

Добавление RAG к одной и той же локальной модели изменило исходное совпадение с 2/30 до 20/30, то есть на 60 процентных пунктов в данном наборе. Правильный документ вошёл в top 3 для всех вопросов, но десять ответов не содержали ожидаемого факта. Следовательно, полнота retrieval не равна правильности генерации.

Облачный исходный результат 29/30 существенно выше принятого 16/30. В тринадцати содержательно подходящих случаях действовавший контракт отклонил Talk с target Player, поскольку требовал пустую цель. В локальных группах встречались MoveTo в Editor вне NavMesh. Эти случаи показывают роль согласования модели и исполнителя: формальное ограничение защищает состояние, но может снизить доступность подходящего ответа.

После опыта политика Planning и подсказки уточнялись. Сохранённые ответы не пересчитывались как новые результаты улучшенного кода. Для оценки изменения конечной точности нужен повтор с тем же корпусом и новым паспортом версии. Выявленное расхождение между исходной и принятой речью остаётся результатом исследования, а не скрывается итоговым процентом.

4.4 Задержки и детерминированная генерация

Для задержки используются медиана и 95-й перцентиль с линейной интерполяцией по отсортированным наблюдениям. Полное время включает подготовку, retrieval и ожидание квоты, а провайдерское — более узкое сетевое обращение. Отсутствующее время провайдера при ошибке не приравнивается к нулю. Различия моделей, оборудования сервера и квот не позволяют объяснять разницу одной причиной.

Измеренные задержки опыта 1.0 приведены в таблице 5.

Таблица 5 — Задержки модельных запросов в опыте 1.0

Конфигурация опыта 1.0	**p50 полного запроса, с**	**p95 полного запроса, с**	**p50 провайдера, с**
Qwen3.5 2B без RAG	6,33	7,33	6,32
Qwen3.5 2B с RAG	6,95	7,92	6,60
GPT-OSS 20B с RAG	30,95	74,54	10,20

В этом историческом опыте Pollinations вызывался последовательно с интервалом 31 секунда. Время ожидания включено в полную задержку. Эта настройка не описывает последующий адаптер 1.1 с клиентским запасом до 46 секунд. Нулевая оплата API в опыте не означает отсутствия стоимости компьютера, электроэнергии или разработки.

Производительность Terrain измерена отдельно от модели. На каждом из разрешений 65, 129, 257 и 513 выполнено десять seed. Замер охватывает TerrainGenerator.Generate, но не создание ассетов, импорт материалов и формирование TerrainSpec. Каждая спецификация сгенерирована дважды, массивы сопоставлены поэлементно.

Время работы генератора приведено в таблице 6.

Таблица 6 — Время генерации массива высот

Разрешение	Число seed	Медиана Generate, мс	p95 Generate, мс
65	10	2,237	3,846
129	10	8,718	8,756
257	10	34,223	36,445
513	10	136,130	137,208

Все 40 повторных пар совпали в проверенной среде. Это подтверждает воспроизводимость алгоритма при одинаковых входах на данной машине, но не между любыми платформами и компиляторами. Рост времени согласуется с числом ячеек n^2. Он не характеризует полное время создания пригодного для игры мира.

4.5 Модельные задания редакторского агента

Редакторский процесс 1.1 проверен семью последовательными заданиями в одной беседе. Автоматический runner читает реальные компоненты и сохраняет ответы. Черновик C# перехватывается для просмотра, но не импортируется автоматически. Результат зависит от накопленного состояния этой беседы, поэтому семь случаев не являются семью независимыми повторениями одной задачи.

Задания и предметные условия приведены в таблице 7.

Таблица 7 — Семь заданий редакторского агента 1.1

№	Проверяемое задание	Предметное условие
1	Создать лесной мир	Terrain и заданные логические объекты действительно созданы

№	Проверяемое задание	Предметное условие
2	Изменить существующий рельеф	Параметры обновлены без создания второго мира
3	Создать NPC	Личность, компоненты и индивидуальные знания назначены
4	Настроить патруль	Встроенное поведение и точки сохранены; это ещё не проверка полного обхода
5	Добавить три источника света	Три Point над поверхностью, нужные цвет, range и интенсивность
6	Подготовить вращающий компонент	Исходник предложен для просмотра, автоматический импорт запрещён
7	Объяснить состояние без изменений	Ответ соответствует контексту, мутаций нет

Локальный прогон qwen3.5:9b по усиленным критериям завершён 31.10.2026 в 12:33:42 UTC на checkpoint 0aa9bdf: 7/7 автоматических случаев. Время отдельных заданий составило от 109,019 до 230,891 с. Световые источники получили зазор приблизительно 3 м над поверхностью и range 20. Источник chat-agent-acceptance-local9b-v2.json сохранён с SHA-256 9beccb5cb4fb332141075749a561760238208bc54841091e31e87d1280751936.

Результат локального прогона следует читать в пределах таблицы. В частности, случай патруля проверяет конфигурацию, а не длительный обход препятствий. Успех исходника в шестом случае означает готовность к просмотру. Фактическая компиляция проверена отдельным сценарием ниже. Результат 7/7 не является общей оценкой надёжности модели на любых редакторских заданиях.

Для бесплатной облачной конфигурации итоговый raw-протокол checkpoint 60192a9 завершён в 12:58:31 UTC: 6/7. В седьмом случае лексический критерий не распознал слово «террейн», хотя количества и модель названы правильно и изменений сцены нет. Независимая проверка подтвердила семь ограниченных условий сценария, но обнаружила дополнительное неподтверждённое обобщение, что все объекты находятся в пределах координат: камера расположена за границей Terrain. Поэтому весь ответ не объявляется полностью достоверным, а автоматический итог остаётся 6/7 случаев.

Различие автоматической и независимой оценки проявилось и раньше. Первый локальный 9B-прогон получил 7/7 по узкому критерию, однако два источника света находились под Terrain. Предложенный вращающий C# содержал неверный overload Rotate и деление на deltaTime. Он не импортировался в Unity; позднейшая изолированная проверка mcs на установленных Unity-сборках подтвердила ошибки CS1502/CS1503. Этот raw-протокол сохранён без изменения. После исправления размещения и усиления критериев выполнен новый локальный v2, а не переименован прежний успех.

4.6 Проверка программного кода и восстановления

Для исходника из успешного локального v2 выполнен отдельный просмотр и проверена привязка к точному hash. Компонент WorldWeaverSmoothRotateY использует допустимый Unity API и изменение угла 30 градусов в секунду через умножение на Time.deltaTime. Затем настоящий Unity-процесс импортировал, скомпилировал, подключил и восстановил файл. Протокол chat-code-acceptance-local9b-v2.json содержит 11/11 выполненных проверок и подтверждение восстановления исходного состояния.

Этот сценарий не означает автоматической безопасности любого модельного кода. Он подтверждает жизненный цикл конкретного просмотренного исходника. Для нового облачного draft отдельно выполнена статическая компиляция mcs с нулевым кодом завершения и корректным

контрольным примером. Нового Unity import/attach/revert для этого облачного исходника не проводилось, поэтому результат локального 11/11 ему не приписывается.

Журнал двух импортов проверен на синтетических заранее просмотренных версиях одного компонента. После A импортируется B, затем проверяются отказ устаревшего attach, запрет возврата A поверх B и корректный порядок возврата B → A. Протокол содержит 34/34 условия и четыре реальные компиляции. Синтетический fixture испытывает сохранность версий; его нельзя суммировать с одиннадцатью проверками модельного исходника как число независимых успешных генераций.

Чистая установка 1.1 проверена в новом проекте-потребителе. Сопоставляются файлы пакета, ссылки ассетов, GUID и отсутствие ошибок компиляции. Это проверка поставки, а не доступности API и не полноты свободного задания. После изменения кода новой редакции чистая установка должна повторяться для нового tarball.

4.7 Программные и предметные проверки редакции 1.2

Для новой редакции сохранён неизменяемый программный срез SEVENTH-r3 из 396 файлов. Его сводный SHA-256 — f6756356eb56ea7f73ecf693f2826c588b7ee97abe1ca45589de028cfdf593aa; commit — ee8d9329a53b416b53ec8f0546cd906263abb331. Манифест перечисляет файлы, а integration-1.2-seventh-r3-summary.json связывает с ними действительные XML. Внешний inference в этой серии отключён: WW_REAL_TESTS равен 0. Таким образом, успешные программные тесты не считаются успешными ответами LLM. Отдельный модельный разговор проведён на R2 и сохраняет собственную привязку к исходникам.

Подтверждённые проверки среза и отдельные предметные испытания сведены в таблице 8. Разные строки имеют разный знаменатель; их результаты не складываются в общий процент качества модели.

Таблица 8 — Подтверждённые программные и предметные проверки редакции 1.2

Проверка	Наблюдаемый результат	Граница вывода
EditMode на неизменном срезе R3	397/397, 0 ошибок и пропусков; в XML 21,903 с	Перечисленные программные свойства, без LLM
PlayMode на том же срезе	9/9; в XML 12,649 с	Конкретные сценарии навигации, игрового состояния и расписания
Подключения моделей	14/14, в том числе реальный domain reload	Входят в 397; HTTP контролируемого сервера, без внешнего inference
Исправление исходников и возврат	Ошибка компилятора, ограниченное исправление, реальный Play-тест и 6 reload	Заранее подготовленные планы, без LLM
Экспорт схемы инструментов	46 инструментов, 47 ветвей; 33/33 проверок; Unity exit 0	Протокол и соответствие исходникам, не семантика ответа модели
macOS Player	Development StandaloneOSX, exit 0; приложение запущено, доставка завершена	Один ручной сценарий; экранные Save/Load и другие платформы не проверены
Ручная проверка игрового ввода	Письмо доставлено; получено 5 монет; повтор E не добавил награду	Одно наблюдение в Unity Play Mode, без LLM

Проверка	Наблюдаемый результат	Граница вывода
Чистая установка R3	Новый проект; tarball 1.2.0; exit 0, ошибок компилятора нет	Проверенная среда Unity 6000.3.27f1, без вывода о всех конфигурациях
Реальный visual-запрос	Строгий критерий формы не пройден; 58,203 с	Верные стороны, цвета и сфера; куб назван более общим термином

В EditMode входят контроль checkpoint и ручных конфликтов, выбор предметных навыков, типизированная схема конкретного обращения, права Ask/Plan/Execute, проверка проектного контекста и потокового протокола. Например, сохраняется исходное условие количества объектов, неизвестные поля отклоняются до изменения сцены, а неоднозначная цель не выбирается произвольно. Индекс проекта проверен на тысяче объектов с повторным чтением после reload. Файл навыка остаётся справочным материалом и не добавляет себе доверие либо права выполнения.

Девять PlayMode-проверок охватывают прежнюю передачу предмета и движение, именованный патруль по полному пути вокруг препятствия, отказ от движения по недостижимому пути, дисковое восстановление доставки, награды и scoped-памяти. Отдельно проверены интервалы расписания с переходом через полночь, реальный переход NPC между разными именованными местами, влияние закрытых ворот на путь и сохранение времени и событий. Это проверки заданных сценариев, а не доказательство проходимости любой созданной сцены.

R2 включил ограниченную проекцию контекста, отдельную обработку изменения только цели NPC и усиленную проверку новых источников света. Изменение цели профиля не приравнивается к поручению настроить физическую навигацию. Специализация одной ветви схемы не удаляет независимые инструменты составного запроса. R3 добавил очистку устаревшей

световой привязки и 14 регрессий переходов. При фиксации протокола различий с рабочим пакетом не найдено. Более ранние серии сохранены как история. Нативный интерфейс проверен отдельно в широком и узком окнах.

Целевой запуск R3 повторил 20 выбранных проверок без ошибок; они входят в 397 и не увеличивают число уникальных случаев. Ранее R2 отдельно повторил 226 случаев своего набора из 383. В детерминированных примерах полный host JSON сцены содержал 30000 символов, проекция для модели — 3107; для задания — 19217 и 1132; для read-back — 35509 и 2720. Значения «до» относятся к полным внутренним данным, а не к точному prompt прежней реализации. Это не измерение токенов, скорости модели или качества ответа. Полное состояние хоста сохраняется; сведения, не вошедшие в проекцию, запрашиваются отдельными инструментами.

В семи фиксированных формулировках только первый запрос требует нового мира. Добавление NPC или трёх источников света «в долине» не должно включать ограничение одного макродействия создания Terrain. Общий конечный предикат DirectWorldCreation используется и выбором инструментов, и уточнением схемы. В целевой проверке контроллер с подготовленным провайдером действительно создал три Point-света одним пакетом без нового мира. Это проверка маршрутизации и исполнителя; самостоятельный выбор тех же действий реальной моделью ею не доказан.

Сквозной инженерный сценарий workflow-focused-verification.json использует заранее подготовленные планы для двух существующих исходников. Первый просмотренный patch намеренно содержит неизвестное имя; Unity возвращает настоящую ошибку. Исправление ограничивается ранее просмотренным файлом и допустимой диагностикой. После успешной компиляции и reload контроллер запускает реальный PlayMode-тест получения урона и смерти, затем возвращает исходное состояние по checkpoint. Зафиксированы шесть перезагрузок домена. Этот опыт проверяет механизм продолжения и восстановления; вывод о способности модели самостоятельно написать верное исправление из него не следует.

Сборка Player выполнена в отдельном проекте-потребителе. Сохранены манифест выходного приложения и hash исполняемого файла. Затем запущено именно это самостоятельное приложение: ввод WASD переместил игрока, E у жителя выдал письмо, а E у стража завершил доставку и выдал пять монет. Повтор E сохранил пять монет. Наблюдение связано с тем же executable SHA в player-native-smoke-1.2.json. Кнопки Save/Load и изменения времени мышью в этом ручном прогоне не подтверждены; автоматические проверки дискового сохранения имеют отдельные доказательства. Подпись, нотариализация macOS, другие платформы и общая производительность приложения не испытывались.

Окончательный tarball 1.2.0 установлен в новый проект-потребитель. Протокол clean-install-1.2.0.json фиксирует exit 0 и отсутствие ошибок компилятора, загрузку стилей чата и схем, действительную команду с Undo. Дважды созданная demo получает разные пути; первая сцена, порядок и флаги Build Settings сохранены. Проверены ссылки Terrain, обновление alphamap, знания и модельные маршруты NPC. Это проверка поставки R3, а не ещё один модельный разговор.

Ручное наблюдение с клавиатурой дополняет программные тесты. Игрок получил письмо у жителя и перенёс его стражу; повторное нажатие E после завершения не создало второй награды. Состояния до и после показаны на рисунке 11 и рисунке 12 в приложении Б.2. Это демонстрационный сценарий с детерминированными правилами, без модельного запроса.

4.8 Приёмочные модельные прогоны редакции 1.2

Ранние незавершённые прогоны сохранены: три и два отказа по времени, затем один случай Balanced с уже созданным миром и не успевшим проверочным ответом. Пятый срез выполнил два задания Deep, после чего разговор остановлен для исправления маршрутизации света. Эти неполные беседы не получают знаменатель семи выполненных задач. Подробные исходы приведены в приложении Б.5.

Полный SIXTH прошёл 5/7 автоматических условий. Цель NPC не обновилась после тайм-аута; три новых Point оказались под Terrain. Условие

наличия draft прошло, но последующее реальное вращение дало 1,051° вместо 60,213° за 2,007 с. Также обнаружены незапрошенное покачивание NPC и неподтверждённые фразы о воде и маршрутах. Поэтому 5/7 не означает пять полностью правильных результатов. Ошибки и исходники сохранены до исправлений.

После исправлений на неизменном R2 проведён новый полный разговор qwen3.5:9b в режиме Deep: с 23:22:58 до 23:43:43 UTC 31.10.2026. Протокол chat-agent-acceptance-local-1.2-final.json содержит 7/7 положительных автоматических исходов; его SHA-256 — 1cfb0fbde6498f83f198a5a7a9bd647cdfcb9fdd57414be35d0a1d670e17c8d9. Этот результат относится к commit 74ceb3f2312966c88ac9a74d2210788290565226 и перечисленным условиям. R3 изменил только исполнение переходов световой привязки и тесты; Runtime, prompt и схема не изменены. Сохранённые команды создавали новые Point с on_terrain:true и не затрагивали исправленную ветвь. Это статическая сверка trace и diff, а не фактическое воспроизведение планов или новый inference на R3.

Условия R2 и полное время задания приведены в таблице 9.

Таблица 9 — Один приёмочный разговор на срезе SEVENTH-r2

№	Задание R2	Проверенный результат	Время, с
1	Создать долину	Terrain 160 × 160; 12 деревьев, 3 дома, 4 камня; свет и камера	182,446
2	Уменьшить высоту	height 24; один мир; прежние количества сохранены	173,033
3	Создать Игната	Страж, осторожная личность и индивидуальный факт о воротах	177,687

№	Задание R2	Проверенный результат	Время, с
4	Изменить только цель	Поле goals обновлено, прежние сведения сохранены; физический обход не поручался	167,587
5	Добавить три Point	Цвет и количество; границы Terrain, зазор над землёй/водой, достаточный range; солнце сохранено	321,562
6	Подготовить C#	Draft сохранён для просмотра; импорта и исполнения runner не выполнял	121,124
7	Объяснить сцену	Указаны модель и состав; состояние и ожидающий draft не изменены	100,332

Пятый случай содержит наблюдаемое исправление самой моделью. Первый план указал spot_angle = 180 при разрешённом диапазоне от 1 до 179 и был отклонён до мутации. В следующем обращении модель исправила значение на 179; исполнитель создал три источника, независимые пространственные условия прошли, затем получен итоговый ответ. Это реальный пример параметрического ремонта после диагностики. Он отличается от сценария с заранее подготовленными планами компиляторного исправления и не подтверждает полное соблюдение числовых ограничений серверным декодером.

Шестой случай оставил ожидающий исходник SmoothYRotationMonoBehaviour. Точный SHA — 2468b871f20cddd64ad5431d897226352a35b6df8992d8cd374b2393bf7e4918; просмотр обнаружил корректное выражение 30 × Time.deltaTime для принимающего градусы Rotate. Сам runner не импортировал код, поэтому его 7/7 не является доказательством компиляции или измеренной скорости.

После просмотра на R3 выполнен отдельный протокол chat-code-acceptance-local-1.2.json: 11/11 условий импорта, настоящей компиляции Unity, подключения компонента и возврата прошли. Новый вызов модели не выполнялся. Импортируемый файл дополнен служебным комментарием происхождения и завершающим переносом; его SHA dbb645e834e447fa398d9bf81eba05ac9c6222ef6f8157228077093711701814 отличается от hash исходного draft по этой причине. После возврата файл отсутствует, повторная компиляция не имеет ошибок, ручная очистка не требуется.

Тот же импортируемый исходник затем испытан в отдельном реальном Play-измерении на R3. За 2,0023 с по 99 отсчётам получено 60,0681° вокруг Y при ожидаемых 60,0678°, отклонение по другим осям равно нулю. Неподвижный отрицательный контроль дал 0° и был правильно отклонён. SHA файла до и после совпал; Play Mode завершён. Протоколы связаны через code-and-runtime-source-1.2.json: генерация произошла на R2, а компиляция и измерение — на R3, без нового модельного вызова. Это проверка конкретного компонента, а не всех возможных программ модели.

Визуальный маршрут испытан отдельно на одном настоящем Camera.Render: слева красный куб, справа синий шар. Перед inference объекты удалены из рабочей сцены; текст запроса не называл правильные формы и цвета. Реальная qwen3.5:9b ответила за 58,203 с: «Слева изображён красный параллелепипед, справа — синий шар». Сцена осталась пустой. Предзаданный строгий критерий слова «куб» не прошёл, исходный протокол имеет passed = false и Unity exit 1. Независимый просмотр подтвердил стороны, цвета, общую коробчатую форму и сферу, но не уточнение куба. Поэтому опыт показывает содержательный ответ по изображению и ограничение точности формы, а не полную успешность visual-приёмки. Повтор для подбора положительного результата не выполнялся.

Новые измерения выполнены на Apple M4 с 16 ГиБ памяти при подключённом питании. Системный lowpowermode равен 1 в профилях

аккумулятора и сети; настройки не изменялись для опыта. Для Ollama зафиксированы Flash Attention, KV-кэш q8_0, один параллельный запрос и не более одной загруженной модели. Deep ограничен шестью раундами, бюджетом задания 480 с и отдельного обращения 240 с. Контекст, бюджеты и нагрузка отличаются от 1.1, поэтому отношение длительностей не трактуется как контролируемое сравнение скорости.

4.9 Ограничения вывода и дальнейшая проверка

QA-корпус содержит 30 коротких вопросов к одному миру и не является скрытым независимым набором. Он недостаточен для оценки художественной последовательности персонажей, отказа на неизвестный факт и устойчивости к атакующим инструкциям. Полезность для начинающих разработчиков и уменьшение ручной работы не измерялись пользовательским исследованием. Документальное сопоставление аналогов также не является их очным испытанием.

Основные показатели 1.1 и R2 относятся к одному конечному разговору каждого указанного среза. Их условия различаются между версиями адаптера и кода. Поэтому локальные 7/7 и облачные 6/7 нельзя использовать как статистическое доказательство превосходства модели. Дополнительные неподтверждённые фразы оцениваются отдельно от узкого критерия завершения.

Те же семь сценариев использовались при поиске и исправлении ошибок. Последующий полный повтор поэтому является приёмочным воспроизведением известных условий, а не независимым испытанием на закрытом наборе. История попыток позволяет проверить происхождение исправлений, но не устраняет влияние выбора сценариев на результат. Для оценки переноса на новые задачи нужен отдельный набор формулировок и миров, не использованный при отладке.

Программные испытания 1.2 подтверждают перечисленные условия состояния задания, checkpoint, продолжения и игрового цикла. Они не устанавливают успешность свободного модельного планирования. Для

пользовательского задания нужно отдельно сопоставить исходную формулировку, принятые планы и конечное состояние; простое наличие компонентов или положительный EditMode не заменяет такой проверки.

Отрицательные случаи включают неоднозначную цель, повреждённое сохранение, конфликт ручной правки и остановку после применённого шага. Итог R2 не переносится на последующую реализацию. Программный срез, статическая сверка планов, компиляция, исполнение и визуальный запрос имеют отдельные доказательства; vision ограничен одним изображением.

ЗАКЛЮЧЕНИЕ

Разработан и испытан WorldWeaver — подключаемый инструментарий создания и изменения динамического игрового мира в Unity по текстовому запросу. Анализ предметной области позволил связать в одном процессе подготовку рельефа, объектов, персонажей и взаимодействий. Работа имеет инженерный характер: её результатом является согласованная реализация, а не новый алгоритм языковой модели или шума.

Обосновано разделение языкового планирования и проверяемого исполнения. Созданы типизированные инструменты, ограниченный контекст проекта, состояние задания и независимые предметные проверки. Для программных изменений предусмотрены просмотр точного исходника, настоящая компиляция Unity и восстановление с контролем конфликтов. Компиляция не считается sandbox или доказательством правильности поведения.

На корпусе из 30 вопросов индивидуальный RAG изменил совпадение ожидаемого факта у одной Qwen3.5 2B с 2/30 до 20/30; в принятой реплике факт сохранился в 16/30 случаях. Облачная конфигурация дала 29/30 исходных и 16/30 принятых совпадений. Расхождение показало влияние прикладного контракта на результат, доступный игроку. Эта метрика не оценивает достоверность всех утверждений.

Генератор воспроизвёл массивы высот для 40 повторных пар в испытанной среде. Окончательный программный срез SEVENTH-r3 прошёл 397/397 EditMode и 9/9 PlayMode-проверок. Сквозной опыт с подготовленными планами подтвердил ошибку компилятора, ограниченное исправление, Play-тест урона и смерти и возврат через шесть reload. Эти проверки характеризуют программные механизмы, а не качество свободного планирования модели.

Один разговор на отдельном R2 прошёл семь приёмочных условий. Модель исправила отклонённый числовой параметр света после диагностики. Его просмотренный C# затем прошёл на R3 11 условий импорта, компиляции, подключения и возврата без нового inference. Реальное измерение дало 60,0681°

за 2,0023 с при требовании 30°/с; неподвижный контроль отклонён. Эти результаты относятся к конкретному исходнику и версиям.

Чистая установка R3 подтверждена в новом проекте. Собран и запущен macOS Player: перенос письма завершился однократной наградой в пять монет. Отдельный visual-запрос правильно связал стороны и цвета, но назвал куб более общим термином и не прошёл строгий критерий формы. Ранние неуспехи, включая полный SIXTH 5/7, сохранены.

Повторяемые семь сценариев использовались при отладке и не являются закрытым независимым корпусом. Превосходство над другими средствами и экономия труда пользователей не доказаны. Для расширения выводов нужны новые миры и формулировки, повторные прогоны и пользовательское исследование. Практический вывод работы — каждое заявление о выполнении должно связываться с наблюдаемым состоянием Unity; полнота задания, сохранность данных и игровое поведение требуют отдельных проверок.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Shaker, N. Procedural Content Generation in Games / N. Shaker, J. Togelius, M. J. Nelson. — Cham : Springer, 2016. — XVI, 237 p. — DOI 10.1007/978-3-319-42716-4.
— URL: https://www.pcgbook.com/ (дата обращения: 30.10.2026). — Текст : электронный.

2. Unity Technologies. TerrainData SetHeights. — [б. г.].
— URL: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/TerrainData.SetHeights.html (дата обращения: 30.10.2026). — Текст : электронный.

3. ReAct: Synergizing Reasoning and Acting in Language Models / S. Yao, J. Zhao, D. Yu [et al.]. — arXiv:2210.03629v3, 2023. — DOI 10.48550/arXiv.2210.03629.
— URL: https://arxiv.org/abs/2210.03629 (дата обращения: 30.10.2026). — Текст : электронный.

4. Ollama. Structured outputs Ollama. — [б. г.].
— URL: https://docs.ollama.com/capabilities/structured-outputs (дата обращения: 30.10.2026). — Текст : электронный.

5. JSON Schema. JSON Schema Draft 2020 12. — 2020.
— URL: https://json-schema.org/draft/2020-12 (дата обращения: 30.10.2026). — Текст : электронный.

6. Lewis P., Perez E., Piktus A. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. — 2020.
— URL: https://arxiv.org/abs/2005.11401 (дата обращения: 30.10.2026). — Текст : электронный.

7. Park J. S., O'Brien J. C., Cai C. J. et al. Generative Agents Interactive Simulacra of Human Behavior. — 2023.
— URL: https://arxiv.org/abs/2304.03442 (дата обращения: 30.10.2026). — Текст : электронный.

8. Unity Technologies. NavMeshAgent. Unity6.3 ScriptReference. — [б. г.]. — URL: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/AI.NavMeshAgent.html (дата обращения: 01.11.2026). — Текст : электронный.

9. Unity Technologies. Assistant modes and model tiers. Assistant 2.21.0-pre.2. — [б. г.]. — URL: https://docs.unity3d.com/Packages/com.unity.ai.assistant@2.21/manual/about/assistant-modes.html (дата обращения: 31.10.2026). — Текст : электронный.

10. Unity Technologies. Use Assistant tools. Assistant 2.21.0-pre.2. — [б. г.]. — URL: https://docs.unity3d.com/Packages/com.unity.ai.assistant@2.21/manual/assets/assistant-tools.html (дата обращения: 31.10.2026). — Текст : электронный.

11. Unity Technologies. Configure AI Gateway Assistant 2.21. — [б. г.]. — URL: https://docs.unity3d.com/Packages/com.unity.ai.assistant@2.21/manual/integration/ai-gateway-get-started.html (дата обращения: 31.10.2026). — Текст : электронный.

12. Unity Technologies. Unity's AI tools meet you where you work. — [б. г.]. — URL: https://unity.com/features/ai (дата обращения: 31.10.2026). — Текст : электронный.

13. CoplayDev. MCP for Unity. — [б. г.]. — URL: https://github.com/CoplayDev/unity-mcp (дата обращения: 30.10.2026). — Текст : электронный.

14. Convai. Configure character actions. — [б. г.]. — URL: https://github.com/Conv-AI/Convai-Documentation/blob/main/plugins-and-integrations/convai-unity-sdk/features/character-actions/configuring-actions.md (дата обращения: 30.10.2026). — Текст : электронный.

15. Inworld AI. Character Studio deprecation and Agent Runtime. — [б. г.]. — URL: https://docs.inworld.ai/character-studio-deprecation (дата обращения: 31.10.2026). — Текст : электронный.

16. NVIDIA. NVIDIA ACE for Games. — [б. г.]. — URL: https://developer.nvidia.com/ace-for-games (дата обращения: 30.10.2026). — Текст : электронный.

17. UndreamAI. LLMUnity. — [б. г.].
— URL: https://github.com/undreamai/LLMUnity (дата обращения: 30.10.2026). — Текст : электронный.

18. Promethean AI. Promethean AI. — [б. г.].
— URL: https://www.prometheanai.com/ (дата обращения: 30.10.2026). — Текст : электронный.

19. Procedural Worlds. Procedural Worlds. — [б. г.].
— URL: https://www.procedural-worlds.com/ (дата обращения: 30.10.2026). — Текст : электронный.

20. Kyzr O. Procedural Generation of Outdoor Scenes Bachelor Thesis. — 2023.
— URL: https://dcgi.fel.cvut.cz/en/theses/2023/kyzrondr/ (дата обращения: 31.10.2026). — Текст : электронный.

21. Marcher F. Procedural Terrain Generation Tool in Unity Master Thesis DOI 10.3217/rb5eq-fv649. — 2024.
— URL: https://repository.tugraz.at/publications/rb5eq-fv649 (дата обращения: 31.10.2026). — Текст : электронный.

22. Wakeford O. R. J. Procedural Terrain Generation with Fixed Points Bachelor Thesis. — 2025.
— URL: https://dspace.cuni.cz/handle/20.500.11956/202622 (дата обращения: 31.10.2026). — Текст : электронный.

23. Aslan P. Understanding LLMs for Game NPCs An Exploration of Opportunities Master Thesis. — 2024.
— URL: https://kth.diva-portal.org/smash/get/diva2%3A1938971/FULLTEXT01.pdf (дата обращения: 31.10.2026). — Текст : электронный.

24. Unity Technologies. Introduction to Git dependencies. Unity 6.3 Manual. — [б. г.].
— URL: https://docs.unity3d.com/6000.3/Documentation/Manual/upm-git.html (дата обращения: 01.11.2026). — Текст : электронный.

25. LangChain. LangChain knowledge base. — [б. г.].
— URL: https://github.com/langchain-ai/docs/blob/main/src/oss/langchain/knowledge-base.mdx (дата обращения: 30.10.2026). — Текст : электронный.

26. LlamaIndex. Using VectorStoreIndex. — [б. г.].
— URL: https://developers.llamaindex.ai/python/framework/module_guides/indexing/vector_store_index/ (дата обращения: 30.10.2026). — Текст : электронный.

27. ggml-org. llama.cpp. — [б. г.].
— URL: https://github.com/ggml-org/llama.cpp (дата обращения: 30.10.2026). — Текст : электронный.

28. ggml-org. llama.cpp GBNF Guide JSON Schema conversion and known limitations. — [б. г.].
— URL: https://github.com/ggml-org/llama.cpp/blob/master/grammars/README.md (дата обращения: 31.10.2026). — Текст : электронный.

29. Unity Technologies. Undo. Unity6.3 ScriptReference. — [б. г.].
— URL: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Undo.html (дата обращения: 01.11.2026). — Текст : электронный.

30. Ollama and Google. EmbeddingGemma Ollama model card. — [б. г.].
— URL: https://ollama.com/library/embeddinggemma (дата обращения: 30.10.2026). — Текст : электронный.

31. Google. EmbeddingGemma 300m model card. — [б. г.].
— URL: https://huggingface.co/google/embeddinggemma-300m (дата обращения: 01.11.2026). — Текст : электронный.

32. Unity Technologies. Unity 6000.3.27f1 Release Notes. — 2026.
— URL: https://unity.com/releases/editor/whats-new/6000.3.27f1 (дата обращения: 30.10.2026). — Текст : электронный.

33. Nice-Wolf-Studio. Unity Claude Skills. README; commit a149d3e995420432f52b0899092daa4b069f441f. — [б. г.].
— URL: https://raw.githubusercontent.com/Nice-Wolf-Studio/unity-claude-skills/a149d3e995420432f52b0899092daa4b069f441f/README.md (дата обращения: 01.11.2026). — Текст : электронный.

34. Besty0728. Unity-Skills. README; commit fb7a8d4208671bca1d1013b2776eb2e4917f4964. — [б. г.].
— URL: https://raw.githubusercontent.com/Besty0728/Unity-Skills/fb7a8d4208671bca1d1013b2776eb2e4917f4964/README.md (дата обращения: 01.11.2026). — Текст : электронный.

35. OpenAI. Create chat completion. OpenAI API Reference. — [б. г.].
— URL: https://developers.openai.com/api/reference/resources/chat/subresources/completions/methods/create (дата обращения: 01.11.2026). — Текст : электронный.

36. Ollama and Qwen. Qwen3.5 2B Ollama model card. — [б. г.].
— URL: https://ollama.com/library/qwen3.5:2b (дата обращения: 30.10.2026). — Текст : электронный.

37. Pollinations. Pollinations Classic text adapter: limits and request normalization. — [б. г.].
— URL: https://github.com/pollinations/pollinations/blob/73311dad9465c7ddd0ceb9cdc0f2ab8b171f6dbb/text.pollinations.ai/cloudflare-cache/src/origin.js (дата обращения: 31.10.2026). — Текст : электронный.

38. Pollinations. Pollinations Classic API. Documentation at commit 73311dad9465c7ddd0ceb9cdc0f2ab8b171f6dbb. — [б. г.].
— URL: https://github.com/pollinations/pollinations/blob/73311dad9465c7ddd0ceb9cdc0f2ab8b171f6dbb/APIDOCS.md (дата обращения: 01.11.2026). — Текст : электронный.

39. Unity Technologies. NavMeshBuilder BuildNavMeshData. — [б. г.]. — URL: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/AI.NavMeshBuilder.BuildNavMeshData.html (дата обращения: 30.10.2026). — Текст : электронный.

40. Unity Technologies. Unity Test Framework introduction Unity 6.3. — [б. г.]. — URL: https://docs.unity3d.com/6000.3/Documentation/Manual/test-framework/test-framework-introduction.html (дата обращения: 31.10.2026). — Текст : электронный.

41. Росстандарт. ГОСТ 19.201-78 Техническое задание Требования к содержанию и оформлению. — 1978. — URL: https://protect.gost.ru/gost/details/b64061cf-6bb2-4c75-aee9-18bb5e26e554 (дата обращения: 30.10.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А Техническое задание

А.1 Введение

Наименование разработки — программный инструментарий WorldWeaver. Тема ВКР — «Разработка интеллектуального инструментария для автоматизированного создания динамических игровых миров». Область применения — учебные и экспериментальные проекты Unity, в которых требуется совместная подготовка окружения, неигровых персонажей, знаний и взаимодействий.

Инструментарий поставляется как UPM-пакет. Он включает редакторскую и исполняемую части. Основной пользователь — разработчик Unity-проекта. Создаваемая демонстрационная сцена является средством проверки пакета, а не самостоятельным коммерческим игровым продуктом.

А.2 Основания для разработки

Разработка выполняется в рамках выпускной квалификационной работы бакалавра по направлению 09.03.01 «Информатика и вычислительная техника», профиль «Разработка игровых и XR приложений», ВолгГТУ, ФЭВТ, кафедра САПР и ПК. Исполнитель — Коваленко Кирилл Дмитриевич, группа ИВТ-465.

Реквизиты приказа об утверждении темы, руководителя и календарного задания заполняются по официальным документам вуза. Состав документа основан на ГОСТ 19.201-78 [41].

А.3 Назначение разработки

Функциональное назначение — преобразование естественно-языковых запросов в проверяемые операции над игровым миром, создание NPC с индивидуальными знаниями и исполнение ограниченного набора игровых действий. Эксплуатационное назначение — работа в существующем Unity-проекте с возможностью замены модельного провайдера и повторного использования компонентов.

Результат должен оставаться доступным через обычные компоненты и ассеты Unity. Запрещено выдавать текстовое подтверждение модели за доказательство изменения сцены. Пользователь должен различать предложение, применённую операцию, результат компилятора и незавершённое условие.

А.4 Требования к программе

А.4.1 Функциональные требования

Функциональные требования приведены в таблице 10.

Таблица 10 — Функциональные требования

Код	Требование
FR-01	Установка через UPM; основной чат на UI Toolkit; выбор модели, режима и бюджета, история, остановка и текущий этап
FR-02	ILLMProvider, подключение Ollama и OpenAI-compatible через форму URL/ID/формата; явная ошибка без скрытой подмены ответа
FR-03	Отдельные маршруты редакторского чата, Terrain, Dialogue, Planning и embeddings
FR-04	Создание Terrain по частичной проверенной спецификации с фиксированным seed; вода, материалы и окружение
FR-05	Частичное изменение существующего мира с сохранением незапрошенных параметров и повторной проверкой спецификации
FR-06	Создание и изменение NPC с ролью, характером, биографией, целями, знаниями и ограничениями

Код	Требование
FR-07	Индивидуальный RAG: загрузка документов, фрагменты, embeddings, область доступа, top k и переиндексация
FR-08	Различимые постоянные знания, разговорные записи и подтверждённые эпизоды
FR-09	Свободный текстовый диалог с реальным модельным запросом и структурированным NpcDecision
FR-10	Контекст из известных сущностей, инвентаря и состояния, без автоматического раскрытия чужих данных
FR-11	Проверенная передача предмета с изменением обоих действительных инвентарей
FR-12	Расширяемый реестр игровых действий с проверкой разрешений и предусловий
FR-13	Высокоуровневое планирование NPC и движение через NavMeshAgent; ограничения частоты запросов
FR-14	Строгая проверка JSON, неизвестных полей, типов, конечных чисел и предметных диапазонов
FR-15	Trace с версиями, моделью, временем, исходным ответом, преобразованием формы и результатами инструментов
FR-16	Timeout, отмена и ограниченные повторы распознанного временного отказа
FR-17	Последовательный цикл планирования, исполнения, read-back и ограниченного исправления

Код	Требование
FR-18	Полная проверка пакета до первой мутации, включая зависимости на создаваемые ранее в пакете цели
FR-19	Завершение по проверке исходной цели; complete не подменяет фактический контроль
FR-20	Работа с активной сценой, явные цели, маркер происхождения, Undo и очистка только собственных новых ассетов
FR-21	Просмотр точного C# до импорта; исторический снимок не является новым разрешением на применение
FR-22	Различимые запись исходника, настоящий результат Unity compiler и подключение нужного компонента
FR-23	Независимый журнал каждого импорта и восстановление с защитой последующих ручных правок
FR-24	Проверка нового NPC в Play Mode без ручной связи с прежним demo UI
FR-25	Сохраняемое состояние задания: требования, этапы, результаты проверок и незавершённые условия
FR-26	Checkpoint обращения и продолжение после reload без повторного исполнения завершённого шага
FR-27	Выбор небольшого числа предметных навыков с происхождением, лицензией и фиксированной версией
FR-28	Индекс проекта и устойчивые цели для работы после reload; неоднозначность не разрешается угадыванием

Код	Требование
FR-29	Семантические места, дороги, ворота и входы, связанные с геометрией и доступностью маршрута
FR-30	Именованный патруль NPC по проверенному NavMesh-маршруту
FR-31	Управляемый игрок, взаимодействие, доставка и однократная награда по действительному состоянию
FR-32	Дисковое сохранение и загрузка игрового состояния и эпизодической памяти нужного мира
FR-33	Просмотренные изменения существующего исходника с проверкой исходного hash и результата компилятора
FR-34	Прикрепление реальных изображений для поддерживающей vision модели и различимый потоковый статус

FR-25–FR-34 определяют проверяемое расширение редакции 1.2. Их наличие в ТЗ не является свидетельством прохождения приёмки. Полный список поддерживаемых инструментов и их параметры должен соответствовать схеме установленного пакета; произвольные поля и reflection-команды не добавляются как запасной способ исполнения.

А.4.2 Требования к надёжности

Ошибочный план должен отклоняться до изменений. При исполнительной ошибке восстанавливается группа Undo и удаляются только ассеты, созданные данным пакетом. Операции чтения не создают фиктивную отмену. Сохранение сцены выполняется только по явной команде.

До фиксации операция может быть отменена без побочного эффекта. После фиксации результат должен оставаться указанным, даже если отменена

последующая индексация памяти. Статус отмены не должен провоцировать повтор уже состоявшейся передачи предмета.

При восстановлении исходника текущее содержимое должно совпадать с ожидаемой версией после импорта. При конфликте ручной правки требуется контролируемый отказ. Для перекрывающихся импортов сохраняются обе резервные версии и допустимый порядок возврата.

После reload система должна распознавать незавершённую компиляцию и не выдавать отсутствие события за успех. После загрузки сохранения награда не выдаётся повторно. Повреждённый файл не должен молча заменять пригодное состояние пустыми данными.

А.4.3 Условия эксплуатации и технические средства

Основная целевая среда — Unity 6000.3.27f1 или версия, отдельно подтверждённая установочным протоколом. Проверка ядра 1.0 на Unity 2022.3 не подтверждает полную совместимость новых функций. Требуются установленный пакет и доступный endpoint выбранного провайдера. Для локального режима требуется отдельный сервер Ollama с загруженными моделями.

Готовый клавиатурный игрок использует legacy Input Manager. Для демонстрации требуется этот режим либо Both; испытанный проект использует activeInputHandler = 0. Для проекта только с New Input System необходима отдельная интеграция ввода; такая конфигурация не считается проверенной.

Checkpoint мутационного обращения ограничен 20000 файлами, 300 МиБ включая рабочие состояния и 1–8 загруженными сценами. Резервируются Assets, ProjectSettings и манифесты пакетов; восстановление файлов ограничено изменённым множеством с контролем конфликтов. Сцены с baked lighting данным механизмом не поддерживаются. Эти пределы учитываются до начала задачи, а не скрываются общим обещанием совместимости с любым проектом.

Испытательная машина — Apple M4, 16 ГиБ памяти, macOS 15.7.4. Это паспорт проведённых измерений, а не доказанный минимальный аппаратный

порог. Большие модельные запросы выполняются последовательно. Недостаток памяти и недоступность сервера отражаются как ошибки среды.

Работа с бесплатным внешним API не должна требовать пополнения баланса, оформления платной подписки или передачи ключа в исходники. Доступность и квота внешнего сервиса не гарантируются пакетом. Разрешён явный локальный режим.

А.4.4 Информационная и программная совместимость

Данные сцены представлены обычными компонентами и ScriptableObject. Текстовые знания поддерживают заявленные установленной версией форматы. Сериализуемые структуры имеют различимый формат версии. Смена embedding-модели требует совместимого индекса или переиндексации.

Endpoint удалённого сервера использует HTTPS; локальный HTTP ограничивается loopback. Credentials не помещаются в URL, пакет и исследовательские протоколы. Контекст ограничивается необходимыми данными, а scopes проверяются до retrieval.

В UserSettings сохраняются только несекретные сведения подключения и имя переменной окружения. Введённый ключ действует в процессе Unity и не записывается в файл или историю; для постоянного доступа используется внешняя переменная окружения. Проверка соединения должна читать список и свойства моделей без генерации; запрос проверки ответа выполняется отдельной явной командой. Форматы ответа выбираются по возможностям конкретного API. Нативные Anthropic Messages и Gemini в этот контракт не входят.

Новое C# не исполняется в sandbox. Просмотр, статическая проверка и компиляция имеют ограниченную область и не гарантируют безопасности. Для обычных операций используются встроенные инструменты с конечным набором параметров.

А.4.5 Требования к результату и ограничения

Процедурное окружение использует примитивы и собственные материалы; оно не объявляется генерацией художественных ассетов коммерческого уровня. Рельеф воспроизводим при одинаковых параметрах в проверенной среде. Межплатформенная битовая идентичность требует отдельного опыта.

Качество ответа, полнота задания и сохранность проекта оцениваются раздельно. Система не гарантирует отсутствие всех фактических ошибок модели. Текстовые ограничения характера NPC не считаются формально обеспеченными правилами, пока они не представлены детерминированной проверкой.

А.4.6 Маркировка и упаковка

Поставка электронная: архив UPM-пакета, исходники и документация. В package.json указываются имя и версия; для передаваемого архива сохраняется SHA-256. В упаковку не включаются credentials, служебные кэши и пользовательские сохранения. Физическая маркировка носителей не устанавливается.

А.4.7 Транспортирование и хранение

Передача выполняется копированием электронных файлов с проверкой контрольных сумм. Исходники и протоколы сохраняются с версиями. Резервные копии импортов отделяются от распространяемого пакета. Специальные климатические условия для программных файлов не устанавливаются; условия физического носителя определяются его изготовителем.

А.5 Требования к программной документации

Поставка должна содержать пояснительную записку, настоящее ТЗ, инструкции установки и повторения испытаний, описание интерфейсов расширения, перечень известных ограничений и машиночитаемые протоколы.

Полное ТЗ включается в приложение пояснительной записки и отдельно формируется из одного исходного файла.

У каждой внешней зависимости указываются назначение и лицензия. Для включённых навыков сохраняются исходный репозиторий и pinned commit. Исходники продукта и исследования не содержат секретов и не требуют закрытого служебного каталога для воспроизведения документов.

А.6 Технико-экономические показатели

Требование оплаты API в приёмочных опытах — ноль. Локальное выполнение использует оборудование пользователя. Нулевая цена вызова не означает нулевую стоимость владения. Срок окупаемости, процент экономии труда и коммерческий эффект без измерений не устанавливаются.

Для оценки производительности фиксируются полное время задания, время провайдера и число исправлений. Время одной генерации массива не считается временем подготовки готового мира. Сопоставление с аналогом допускается только при одинаковых входах и разрешениях либо явно обозначается как анализ документации.

А.7 Стадии и этапы разработки

Последовательность работ: анализ задачи и аналогов; формирование требований; проектирование контрактов и границ; реализация основы и расширений; программные испытания; реальные модельные сценарии; независимая предметная проверка; чистая установка; подготовка документации. Календарные даты определяются утверждённым заданием на ВКР.

Каждый эксперимент имеет собственный checkpoint. Изменение кода после выявления дефекта требует нового прогона соответствующей проверки. Исторический неуспех сохраняется вместе с первоначальным критерием.

А.8 Порядок контроля и приёмки

Проверки и условия приёмки приведены в таблице 11.

Таблица 11 — Проверки и условия приёмки

Код	Проверка	Условие приёмки
AT-01	Чистая установка	Новый проект и Library, действительные ссылки, ноль ошибок компиляции
AT-02	Провайдеры	Реальные локальный и доступный бесплатный ответы; отсутствие скрытого fallback
AT-03	Terrain	Валидная спецификация и повторный массив при прежнем seed
AT-04	Индивидуальные знания	Ожидаемый документ извлечён, чужой scope исключён
AT-05	Передача предмета	Проверены оба инвентаря до и после
AT-06	Недопустимое действие	Отказ до предметного побочного эффекта
AT-07	Навигация	Полный путь и фактическое перемещение в Play Mode
AT-08	Сетевая ошибка	Ограниченный retry, timeout и cancellation без блокировки
AT-09	Многошаговый цикл	Следующий раунд получает реальные результаты и исходную цель
AT-10	Частичное изменение	Изменён прежний мир, незапрошенные параметры сохранены
AT-11	Неверная поздняя команда	Весь план отклонён до мутации

Код	Проверка	Условие приёмки
AT-12	Undo и rollback	Одна группа отмены; новые ассеты очищены, чужие сохранены
AT-13	Просмотр плана	Без применения сцена не меняется
AT-14	Импорт C#	Точный просмотренный файл, событие компилятора и действительный компонент
AT-15	Восстановление C#	Собственные версии возвращены; ручная правка защищена
AT-16	Типизированная схема	Неизвестные ключи, типы и диапазоны отклонены
AT-17	Отмена вокруг commit	До фиксации нет изменений; после неё результат явно указан
AT-18	Запрос только объяснить	Нет мутации, нового draft или attach
AT-19	Нормализация провайдера	Исходный и принятый ответы различимы; запреты не ослаблены
AT-20	Состояние задания	Исходные требования, этапы и незавершённые условия сохранены
AT-21	Reload и продолжение	Нет повторного применения, статус компиляции подтверждён
AT-22	Навыки	Выбраны релевантные документы; происхождение и лицензии доступны
AT-23	Устойчивая цель	После reload выбран тот же объект; неоднозначность отклонена

Код	Проверка	Условие приёмки
АТ-24	Семантические места	Имена соответствуют геометрии и доступным точкам
АТ-25	Именованный патруль	NPC действительно проходит маршрут по NavMesh
АТ-26	Игровой цикл	Игрок взаимодействует и завершает доставку по действительным данным
АТ-27	Однократная награда	Повтор и Save/Load не создают вторую награду
АТ-28	Дисковое состояние	Инвентарь, этап и scoped-память восстановлены из сохранения
АТ-29	Просмотренная правка	Исходный hash проверен, diff применён и компиляция подтверждена
АТ-30	Визуальный контекст	Передан настоящий снимок, поддержка vision и исход отражены

Результаты АТ-20–АТ-30 относятся к новой редакции и принимаются только по её окончательным протоколам. Модельный сценарий считается выполненным по предметным условиям, а не по одному полю success или фразе «готово». Число повторов, исходные prompts и критерии фиксируются до запуска. Отрицательные исходы входят в отчёт.

А.9 Источники разработки

Используются первичная документация Unity, Ollama, применяемых API, исследования RAG и агентного взаимодействия, JSON Schema и ГОСТ

19.201-78. Библиографические сведения приведены в общем списке пояснительной записки и электронном каталоге.

ПРИЛОЖЕНИЕ Б Материалы повторения проверки

Б.1 Установка и паспорт опыта

Используется отдельный Unity-проект, соответствующий версии проверяемого пакета. Для R3 проверен архив com.worldweaver.toolkit-1.2.0.tgz; его SHA и результат нового проекта сохранены в clean-install-1.2.0.json. До запуска фиксируются Unity patch, ОС, оборудование, Git SHA, версия tarball, модель и её digest, endpoint и параметры inference. Архивы прежних редакций не подменяются новой сборкой. Проверка начинается с новой Library, чтобы исключить зависимость от локального кэша разработчика.

Для готового игрока выбирается legacy Input Manager или Both. Сохранённый проект испытан с activeInputHandler = 0; режим только New Input System требует дополнительного подключения ввода. До мутационной задачи также проверяются пределы checkpoint: до 20000 файлов и 300 МиБ, 1–8 загруженных сцен, без baked lighting.

После добавления пакета открывается основной чат WorldWeaver. Выбираются доступная модель и режим. Для локального варианта предварительно запускается Ollama и проверяется загрузка нужных моделей. Внешний API используется только при фактической бесплатной доступности без покупки токенов. Если сервис недоступен, сохраняется исход ошибки; скрытая подмена модельного ответа запрещена.

Для своего сервера нажимается «Добавить API…». Указываются протокол Ollama либо OpenAICompatible, URL и точный ID модели. Пример локального OpenAI-compatible пути — http://127.0.0.1:11434/v1; авторизация для такого сервера не обязательна. Для внешнего сервера выбирается поддерживаемый им формат JsonSchema, JsonObject либо Text. В поле «Имя переменной» вводится имя заранее заданной переменной окружения, а не значение ключа. Для ключа только на время работы Unity используется отдельное поле «Ключ API»; эти два способа не заполняются одновременно.

После «Добавить и выбрать модель» выполняется «Проверить соединение». Получение списка не запускает inference. Команда «Проверить ответ» используется отдельно и только при допустимости расхода квоты. Отсутствие /models у нестандартного сервера не доказывает отсутствия Chat Completions, поэтому исход проверки и выбранный протокол фиксируются в паспорте опыта. Поддержка собственного API не означает универсальной совместимости со всеми поставщиками.

Перед мутацией сохраняется исходная сцена. Для нового опыта создаётся отдельная тестовая сцена или точный снимок исходного состояния. Нельзя запускать runner, рассчитанный на пустую сцену, поверх несохранённой пользовательской работы. Пути проверяемых ассетов фиксируются до и после, а число ручных действий записывается отдельно от модельного исполнения.

Б.2 Программные и предметные проверки

Программные тесты запускаются средствами Unity Test Framework. В сохранённом скрипте run_unity.py режим edit выбирает EditMode, а play — прежние NavigationTests и AssistantBehaviorTests. Полная проверка 1.2 дополнительно включает WorldGameplayTests и WorldScheduleTests: именно четыре fixture перечислены в integration-1.2-seventh-r3-summary.json. Для повторения полного PlayMode-набора выбирается соответствующий фильтр Unity Test Runner; запуск старого сокращённого фильтра не считается повторением девяти проверок. XML сохраняется до следующего прогона. Наличие успешного XML не означает выполнения всех модельных запросов.

Для редакторского сценария сохраняются исходный prompt, попытки, принятые планы и read-back. Независимая проверка читает компоненты и файлы, не полагаясь на Summary инструмента. Если задания выполняются последовательно в одной беседе, этот факт указывается: позднее задание зависит от ранее созданного состояния.

Проверка мира включает TerrainSpec, размеры, seed и логические объекты. Проверка света включает тип, цвет, интенсивность, range, положение над поверхностью и принадлежность точки Terrain. Для NPC проверяются

настоящие поля личности, KnowledgeBase, scope и маршруты. Наличие компонента patrol не считается доказательством полного прохождения пути.

Для игрового действия сравниваются оба инвентаря до и после. Для движения фиксируются начальная позиция, путь и конечная область. Для нового Save/Load проверяется дисковый файл и восстановленные данные после пересоздания состояния. Повторное завершение доставки проверяется отдельно, чтобы обнаружить дублирование награды.

Сохранённые состояния настоящего игрового интерфейса показаны ниже. Крупный фрагмент оформлен средствами Word; исходные JPEG не перерисовывались.

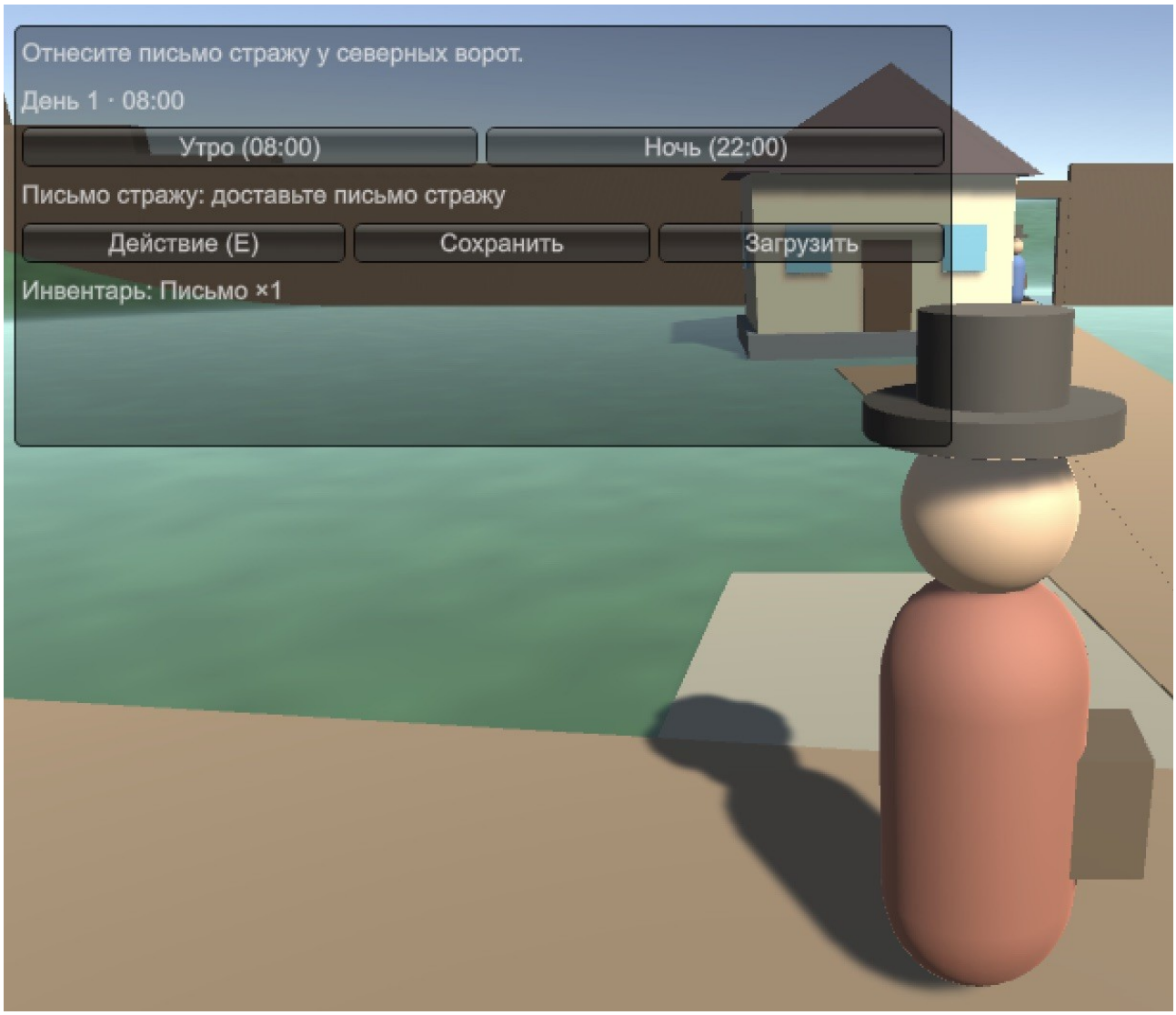


Рисунок 11 — Фрагмент игрового интерфейса после получения письма

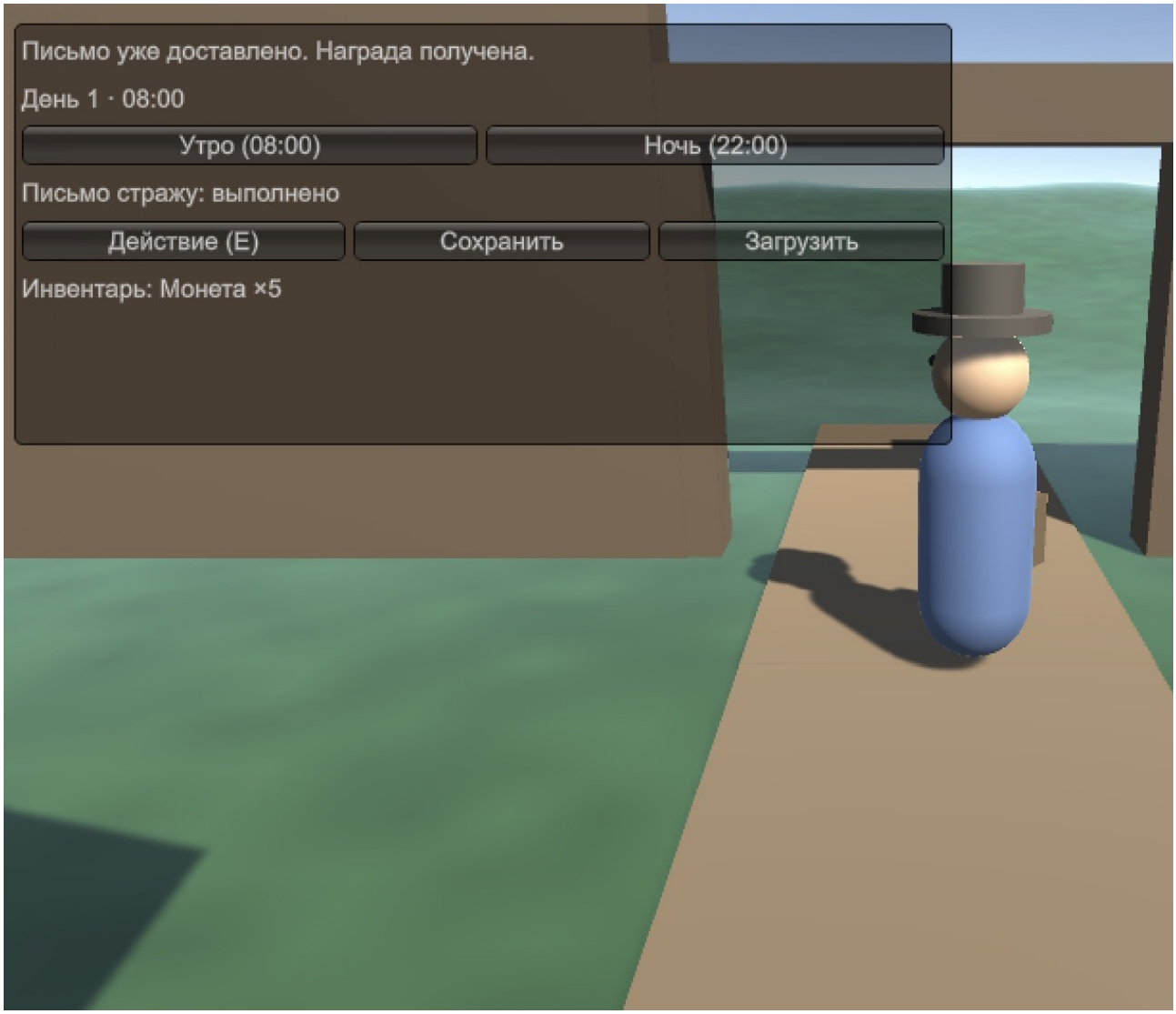


Рисунок 12 — Завершённая доставка после повторного взаимодействия: пять монет

Б.3 Просмотр и компиляция исходника

Модельный draft сохраняется без автоматического импорта. Проверяется точный текст, путь и SHA-256. Вращающий пример должен использовать существующий overload Unity API, правильную ось, единицу угловой скорости и умножение на Time.deltaTime при покадровом обновлении. Статическая компиляция с Unity reference assemblies является предварительным этапом, а не подтверждением исполнения в Unity.

После просмотра запускается отдельный реальный импорт. Запись, событие компилятора, тип компонента и attach проверяются раздельно. Во время reload состояние ожидания сохраняется. После возврата проверяются

исходный файл, .meta, отсутствие лишнего компонента и результат повторной компиляции.

Двухимпортный сценарий использует заранее просмотренные версии A и B. Сначала импортируется A, затем B в тот же путь. Попытка вернуть A поверх B должна быть отклонена. Затем возвращаются B и A, проверяются обе резервные версии. Дополнительно вносится контролируемая ручная правка, после которой автоматическое восстановление обязано отказаться от перезаписи.

Б.4 Исторический корпус индивидуальных знаний

Исходный QA-эксперимент относится к тегу experiment-2026-10-30. Он содержит три группы по 30 запросов без накопления памяти: local-no-rag, local-rag и cloud-rag. Для каждой строки сохраняются ожидаемые варианты ответа, правильный DocumentId, raw trace, принятый результат и время. Неуспешный запрос не удаляется из знаменателя.

Скрипт analyze_results.py агрегирует сохранённые JSONL. Совпадение факта в исходной речи, факт в принятой реплике, JSON-valid, принятие действий и retrieval считаются раздельно. Повтор анализа над теми же строками должен воспроизводить сводку. Повторный inference представляет новый опыт и может дать другой результат.

Содержание корпуса хранится в электронных материалах. Оно является авторским лором вымышленного мира и не предназначено для проверки общих энциклопедических знаний модели. При расширении исследования подготавливается новый независимый набор, включая неизвестные факты, противоречивые документы и отрицательные запросы доступа.

Б.5 Ранние отрицательные наблюдения 1.2

Первые два незавершённых модельных прогона 1.2 сохранены отдельно. В первом записаны три неуспешных задания с ожиданием около 150 с каждое; во втором — два с длительностью около 153 и 242 с. Оставшиеся случаи не завершены. Поздние запросы зависят от не созданного первым запросом

состояния и не являются независимыми повторами. В третьей попытке Balanced один случай завершился лимитом через 240,128 с. При этом в сцене уже находились Terrain 160 × 160 м, 12 деревьев, 3 дома и 4 камня, а массив высот не был плоским. Создание мира состоялось, но завершающий проверочный ответ не уложился в бюджет. Изменение контекста и нагрузка машины различались; единственная причина задержки не установлена.

Последующая попытка Deep на пятом срезе завершила два первых задания с положительными предметными проверками: создание мира за 190,035 с и изменение высоты за 162,315 с. Затем разговор остановлен для исправления маршрутизации запроса о свете. Оставшиеся пять заданий не завершены. Это две успешные выполненные задачи незавершённой беседы, а не итог «2/7» или «7/7».

Полный разговор на шестом срезе завершил все семь заданий и прошёл пять автоматических условий. В четвёртом случае запрос изменения цели NPC прервался через 240,444 с, цель не изменилась. В пятом созданы три Point, но явный on_terrain:false оставил их под поверхностью; прежняя экранная проверка количества ошибочно сообщила положительный итог. При создании NPC также появилось незапрошенное покачивание. В последнем read-only случае сцена сохранилась, однако ответ содержал неподтверждённые заявления о доле воды и назначенных маршрутах. Следовательно, 5/7 не означает пять полностью достоверных ответов.

Шестое условие этого разговора подтверждало только сохранение draft для просмотра. Исходник передавал Mathf.Deg2Rad × 30 в принимающий градусы Transform.Rotate. После просмотра точный неизменённый исходник импортирован в изолированном проекте; реальное Play-измерение дало 1,051° вместо ожидаемых 60,213° за 2,007 с, 99 отсчётов. Неподвижный контроль дал 0°. Протокол rotation-runtime-1.2-sixth-fresh-draft.json сохранил отрицательный исход и совпадение SHA до и после. Наличие draft и успешная компиляция не подтвердили требуемую угловую скорость.

ПРИЛОЖЕНИЕ В Реестр доказательств

Назначение основных протоколов сведено в таблице 12.

Таблица 12 — Реестр доказательств

Материал	**Что подтверждает**	**Что не подтверждает**
XML EditMode и PlayMode	Исходы перечисленных программных проверок	Общую успешность модели
QA JSONL и summary	Измерения фиксированного корпуса 1.0	Качество нового редакторского агента
local9b-v2 acceptance	Семь условий одной локальной беседы 1.1	Успех на произвольных запросах
cloud-v4 acceptance и review	Raw 6/7 и независимый разбор	Полную достоверность каждой дополнительной фразы
local9b-v2 code acceptance	Жизненный цикл одного просмотренного C#	Безопасность любого нового кода
journal acceptance	Сохранность двух синтетических импортов	Число успешных модельных генераций
Clean install	Независимость установки от прежней Library	Доступность внешнего сервиса
Сводка и immutable snapshot 1.2 R3	397 EditMode и 9 PlayMode на перечисленных байтах	Новый inference на R3; разговор 7/7 относится к R2

Материал	Что подтверждает	Что не подтверждает
Workflow 1.2	Компилятор, ограниченный ремонт, Play-тест и checkpoint через reload	Самостоятельное модельное исправление
Разговор R2 и код на R3	7/7 условий; отдельно 11/11 импорта и реальное вращение	Новый inference на R3 или правильность любого кода
Visual 1.2	Строгий критерий не пройден; независимая частичная оценка сохранена	Полную точность формы и общее качество vision
Schema export 1.2	46 инструментов, 47 ветвей и 33 условия протокола	Полноту произвольной цели
Player build и native smoke 1.2	Сборку macOS, запуск, доставку и однократную награду	Экранные Save/Load, другие платформы
Нативный GUI 1.2	Компоновку, API-форму и ручную доставку письма	Модельную генерацию сцены

Новые результаты 1.2 включаются в реестр только после фактического завершения и привязки к версии кода. Подлинная справка проверки заимствований оформляется средствами, принятыми вузом; локальный отчёт не заменяет её и не содержит вымышленного процента оригинальности.